

Đặc trưng và ghép cặp học được (Learned Features and Matching)

PHỤ THUỘC F/27-dat-hoc-sau-vao-dau (khung bốn cổng — chương này là bài tập lớn đầu tiên của khung đó); C/16 cho *attention* (SuperGlue và LoFTR đều là *transformer*); E/24-thi-nghiem-va-ablation cho *oracle* experiment, công cụ quyết định của chương này.

TÓM TẮT

Chương này trả lời một câu hỏi rất hẹp: có nên thay ORB bằng một bộ đặc trưng học được không. Câu trả lời cần ba thứ. Thứ nhất là chẩn đoán: ORB gây ở đúng bốn chỗ, và cả bốn đọc được từ cơ chế của FAST và BRIEF. Thứ hai là bản đồ phương pháp: SuperPoint, R2D2, DISK, XFeat, ALIKED cho đặc trưng; SuperGlue và LightGlue cho ghép cặp; LoFTR và RoMa bỏ hẳn bộ phát hiện. Thứ ba là công đánh giá, và đây là phần quan trọng nhất. Đọc xong bạn nói được bằng cơ chế vì sao ảnh mờ giết bộ *phát hiện* còn đối độ sáng giết bộ *mô tả*, viết được công thức chi phí bậc hai của SuperGlue theo số điểm, và biết chính xác những gì trong ORB-SLAM3 phải bỏ đi để nhận một bộ đặc trưng mới. Nếu chỉ nhớ một điều: tính lặp lại, độ chính xác ghép và tỉ lệ inlier đều có thể tăng mạnh mà APE không nhúc nhích, nên hãy chạy *oracle* ghép cặp *trước* khi tích hợp bất cứ mạng nào.

KIẾN THỨC CẦN CÓ TRƯỚC

- **FAST** — bộ phát hiện góc của ORB. Nó xét 16 điểm ảnh trên một vòng tròn bán kính 3 quanh điểm p , và nhận p là góc nếu có ít nhất 9 điểm liên tiếp cùng sáng hơn $I_p + t$ hoặc cùng tối hơn $I_p - t$, với t là ngưỡng tương phản [1].
- **BRIEF** — bộ mô tả của ORB. Nó lấy 256 cặp điểm cố định trong một ô ảnh nhỏ quanh góc, mỗi cặp trả về một bit “điểm A sáng hơn điểm B hay không” [2]. ORB thêm bước xoay bộ cặp theo hướng của góc.
- **khoảng cách Hamming** — số bit khác nhau giữa hai chuỗi nhị phân. Với ORB thì hai bộ mô tả 256 bit gần nhau nghĩa là hai ô ảnh trông giống nhau.
- **tính lặp lại (repeatability)** — tỉ lệ điểm được bộ phát hiện tìm thấy ở cùng một vị trí trên vật thật trong hai ảnh khác nhau. Đây là thước đo của riêng bộ phát hiện, chưa dính gì tới bộ mô tả.
- **độ chính xác ghép trung bình (MMA)** — trong các cặp được ghép, tỉ lệ cặp có sai số vị trí dưới một ngưỡng điểm ảnh cho trước. Đây là thước đo chung của cả bộ mô tả lẫn bộ ghép cặp.
- **tỉ lệ inlier** — tỉ lệ cặp ghép sống sót sau khi RANSAC ước lượng mô hình hình học. Nó đo mức sạch của tập ghép cặp đưa vào tối ưu.
- **APE** — sai số vị trí tuyệt đối của quỹ đạo so với chân trị, sau khi căn chỉnh hai quỹ đạo. Đây là con số cuối cùng mà một hệ SLAM bị đánh giá, và là thứ duy nhất trong bốn thước đo trên thật sự quan trọng.
- **vận tải tối ưu (optimal transport)** — bài toán tìm cách chuyển một phân phối khối lượng sang một phân phối khác với tổng chi phí nhỏ nhất. Ghép hai tập điểm ảnh với ràng buộc mỗi điểm dùng một lần chính là một bài toán như vậy.

CÂU HỎI MỞ ĐẦU

- Q1. Ảnh mờ giết FAST qua đúng một đại lượng. Đại lượng đó là gì, những tham số nào của hệ bạn quyết định ngưỡng, và bạn đổi ngưỡng đó sang tốc độ quay bằng phép chia nào?
- Q2. BRIEF chỉ so sánh “A sáng hơn B hay không”, nên nó bất biến với mọi phép đổi độ sáng đơn điệu. Vậy vì sao đổi độ sáng vẫn giết nó trong thực tế?
- Q3. SuperGlue tốn chi phí tỉ lệ N^2 theo số điểm. Hệ SLAM của bạn đã có một dự đoán chuyển động từ IMU. Vì sao dự đoán đó biến N^2 thành gần như tuyến tính, và vì sao không paper nào tận dụng nó?
- Q4. Bạn thay ORB bằng SuperPoint và thấy số cặp ghép đúng tăng mạnh, tỉ lệ inlier cũng tăng theo. Vì sao APE vẫn có thể không đổi một milimét nào?
- Q5. Nếu bỏ bộ phát hiện đi thì bức tường trắng có điểm ghép. Vậy vì sao không hệ SLAM thời gian thực nào bỏ bộ phát hiện?

1.1 Đặt vấn đề

Frontend là khối bị đề nghị thay bằng mạng nhiều hơn mọi khối khác trong SLAM. Lý do dễ hiểu. Nó đứng ngoài cùng, nó chạm trực tiếp vào điểm ảnh, và nó là chỗ mà bạn *nhìn thấy* hệ thống chết. Khi camera quay nhanh và tracking mất, thứ hiện ra trên màn hình gỡ lỗi là số cặp ghép tụt từ vài trăm xuống một nhúm. Kết luận trực giác là frontend yếu.

Kết luận đó đúng một nửa. Frontend đúng là yếu ở bốn chỗ cụ thể, và chương này chỉ ra chính xác bốn chỗ đó bằng cơ chế chứ không bằng cảm giác. Nhưng nửa còn lại quan trọng hơn: frontend hiếm khi là ràng buộc chặt nhất của độ chính xác quỹ đạo. Số cặp ghép chỉ tụt xuống mức nguy hiểm trong những khoảng rất ngắn của một chuỗi dài nhiều phút, và tỉ lệ giữa hai khoảng thời gian đó là con số đầu tiên bạn phải tự đo. Trong phần còn lại, ORB đã cho nhiều cặp ghép hơn mức tối ưu hoá cần.

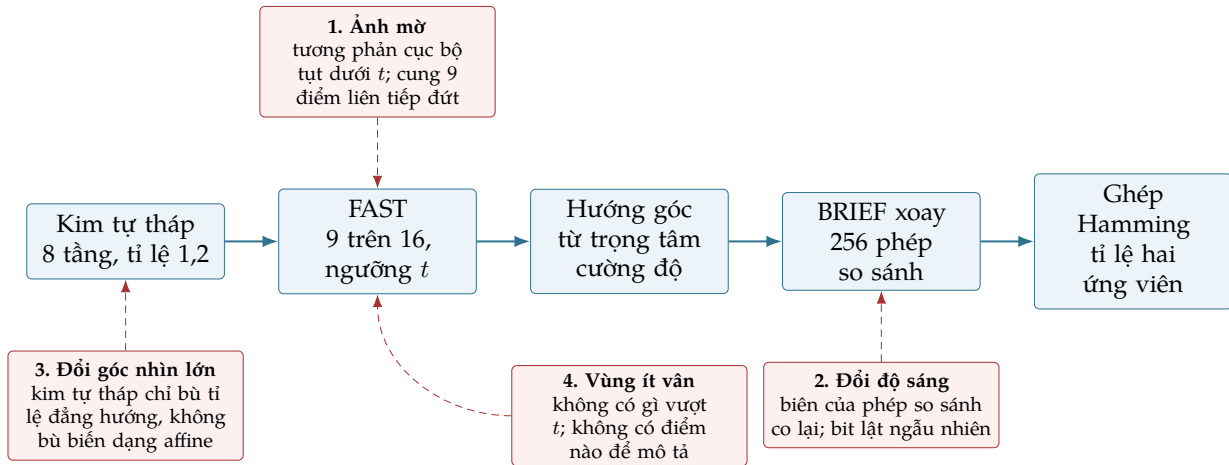
Vì thế câu hỏi đúng không phải “bộ đặc trưng nào tốt hơn”. Câu hỏi đúng là: *trong bao nhiêu phần trăm thời gian của chuỗi, frontend là thứ chặn?* Và trong khoảng thời gian đó, một frontend hoàn hảo thì cứu được bao nhiêu? Chương 24 đã cho công cụ trả lời câu này, đó là *oracle* experiment. Chương này chỉ áp nó vào một khối cụ thể.

Cách trình bày vì thế đi theo thứ tự chẩn đoán rồi mới tới thuốc. Mục 2 mổ xẻ bốn chế độ hỏng của ORB bằng chính công thức của FAST và BRIEF, kèm hai phép tính tay ra con số. Mục 3 và 4 là bản đồ phương pháp, tách làm hai vì đặc trưng và ghép cặp là hai bài toán khác nhau và tiến hoá tách nhau. Mục 5 nói về nhánh bỏ hẳn bộ phát hiện. Mục 6 là cổng đánh giá và là mục nên đọc trước nếu bạn chỉ có mười lăm phút.

→ **tiền đề** F/27 — bốn cổng của chương 27 — bài toán có thật, đo được, vừa ngân sách, hỏng có kiểm soát — là khung mà cả chương này đang điền vào

1.2 Bốn chế độ hỏng của ORB, đọc bằng cơ chế (Four Failure Modes)

ORB là hai thứ ghép lại: một bộ phát hiện góc FAST và một bộ mô tả nhị phân BRIEF có xoay [3]. Bốn chế độ hỏng phân bố không đều giữa hai thứ đó, và biết cái nào hỏng ở đâu quyết định bạn nên thay cái gì. Hình 1.1 đặt cả bốn lên đúng vị trí của chúng trong đường ống.



Hình 1.1. Bốn chế độ hỏng của ORB gắn vào đúng khối mà chúng phá. Hai chế độ đầu tấn công hai khối khác nhau, nên chúng đòi hai thuộc tính khác nhau: mờ giết bộ *phát hiện*, đổi sáng giết bộ *mô tả*. Chế độ 3 là giới hạn của bất biến kim tự tháp. Chế độ 4 không phải lỗi cài đặt mà là hệ quả tất yếu của việc bắt bộ phát hiện quyết định sớm và cục bộ.

A. Ảnh mờ do chuyển động giết bộ phát hiện, và đại lượng quyết định là gì

FAST hỏi một câu hoàn toàn cục bộ: quanh điểm này, có chín điểm ảnh liên tiếp trên vòng tròn bán kính 3 nào lệch hơn ngưỡng t không. Mờ do chuyển động là một phép lấy trung bình dọc theo hướng chuyển động. Trung bình làm giảm biên độ của mọi biến thiên có bước sóng ngắn hơn cửa sổ trung bình.

Mô hình một chiều đủ để thấy hết cơ chế. Gọi cấu trúc ảnh quanh góc có bề rộng w điểm ảnh và biên độ tương phản C mức xám. Làm mờ bằng một cửa sổ dài d điểm ảnh giữ lại xấp xỉ

$$C_{\text{hiệu dụng}} \approx C \cdot \min\left(1, \frac{w}{d}\right).$$

Góc còn được phát hiện chừng nào $C_{\text{hiệu dụng}} > t$. Cho hai vế bằng nhau thì được độ dài vết mờ tới hạn:

$$d^* \approx \frac{wC}{t}.$$

Công thức này nói ba điều, và cả ba dùng được ngay mà không cần một con số nào. Thứ nhất, dung sai mờ tỉ lệ *ngược* với ngưỡng t : hạ ngưỡng thì mua thêm dung sai theo tỉ lệ, chứ không phải bù trừ chút ít. Thứ hai, dung sai tỉ lệ *thuận* với bề rộng cấu trúc, nên cấu trúc thô sống lâu hơn cấu trúc mịn — và đó là lý do khi mờ thì tập điểm còn sót lại toàn góc thô, những góc định vị kém, làm sai số tái chiếu tăng ngay cả khi số cặp ghép chưa tụt. Thứ ba, dung sai tỉ lệ thuận với tương phản, nên cùng một chuyển động sẽ giết bộ phát hiện sớm hơn hẳn trong cảnh thiếu sáng.

Đổi d^* sang đơn vị vật lý cũng chỉ là một phép chia. Một chuyển động quay làm ảnh trượt d điểm ảnh trong thời gian phơi sáng Δt ứng với tốc độ góc

$$\omega \approx \frac{d/f}{\Delta t},$$

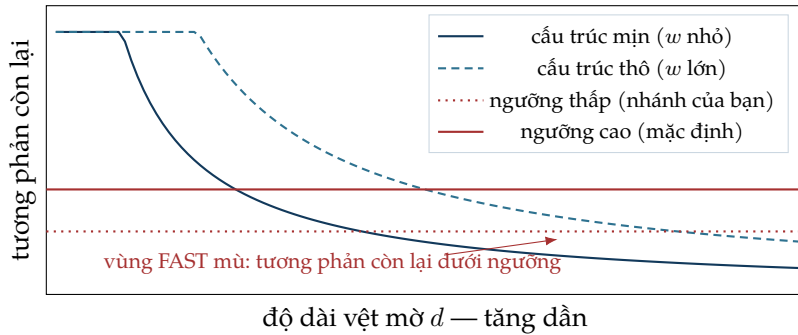
với f là tiêu cự tính bằng điểm ảnh. Ghép hai công thức lại thì ngưỡng tốc độ góc của *hệ bạn* tỉ lệ với $wC/(t f \Delta t)$.

Chương này cố ý dừng ở đó và không đưa một con số. Lý do rất đơn giản: w , C và Δt thật của chuỗi bạn chạy thì mình chưa đo, và một con số chưa đo thì không đáng nằm trong tài liệu học — dán nhãn “ước lượng” cũng không cứu được, vì con số vẫn sẽ được nhớ và được trích lại. Bốn đại lượng ở vế phải đều nằm trong tầm tay bạn: ngưỡng FAST nằm trong cấu hình, tiêu cự nằm

trong file hiệu chỉnh, thời gian phơi sáng nằm trong nhật ký camera, còn w và C thì đo được từ chính ảnh. Thay chúng vào là bạn có ngưỡng của riêng mình trong vòng mười phút, và có nó bằng số đo chứ không bằng lời của ai.

Cái đáng mang đi là một dự đoán định tính, và nó đủ sắc để kiểm chứng được: ngưỡng ấy nằm trong tầm tốc độ quay của một người cầm camera đi bộ bình thường, chứ không phải một tình huống hiểm — nói cách khác, các đợt xoay tay bình thường đã đủ đẩy FAST vào vùng mù, và các chuỗi khó của EuRoC thì vào vùng đó liên tục. Dự án thứ hai ở cuối chương là cách kiểm chứng dự đoán đó trên dữ liệu của chính bạn, và nó chạy trong nửa ngày.

Hình 1.2 vẽ quan hệ đó dưới dạng thuần hình dạng, không một vạch số nào. Điều đáng chú ý nhất trong hình không phải đường cong mà là độ dốc: quanh vùng ngưỡng, giảm t một chút mua thêm được khá nhiều dung sai mờ. Đó chính là lý do hạ iniThFAST từ 20 xuống 12 có tác dụng thật, và cũng là lý do nó có giới hạn — hạ tiếp thì nhiều bắt đầu qua cửa.



Hình 1.2. Tương phản còn lại sau khi làm mờ, theo mô hình trung bình một chiều. Trục cố ý không có vạch số: hình này nói về *hình dạng*, không về một mức mờ cụ thể nào. Ba điều đọc được từ hình: giao điểm với đường ngưỡng là chỗ FAST ngừng thấy góc; hạ ngưỡng đẩy giao điểm sang phải khá nhiều, nên nó có tác dụng thật; và cấu trúc thô sống lâu hơn cấu trúc mịn, nên khi mờ thì các góc còn sót lại đều là góc thô — chúng định vị kém hơn, và sai số tái chiếu tăng ngay cả khi số cặp ghép chưa tụt. Muốn có con số thì thay w, C, t của chính bạn vào $d^* = wC/t$.

Có một hệ quả tinh tế mà mô hình trên chưa nói hết. FAST không đòi biên độ lớn, nó đòi *chín điểm liên tiếp*. Mờ do chuyển động là bất đẳng hướng: nó xóa tương phản theo hướng chuyển động và giữ nguyên theo hướng vuông góc. Nên với một góc, cung liên tiếp bị cắt đứt ở hai đầu trước khi biên độ trung bình chạm ngưỡng. Nói cách khác, FAST chết sớm hơn mô hình một chiều dự đoán, và chết theo cách phụ thuộc vào hướng của cạnh so với hướng quay. Đó là lý do khi quay ngang thì các cạnh dọc biến mất trước, và tập điểm còn lại bị lệch về một hướng — một tập điểm lệch hướng làm ma trận thông tin của bài toán tư thế bị điều kiện xấu, chứ không chỉ làm ít điểm đi.

B. Đổi độ sáng giết bộ mô tả, qua biên chứ không qua giá trị

Đây là chỗ trực giác thông thường sai. BRIEF không lưu giá trị cường độ; nó chỉ lưu 256 kết quả so sánh dạng $I(a) > I(b)$. Một phép biến đổi độ sáng *đơn điệu tăng* và *toàn cục* — chỉnh gamma, đổi thời gian phơi sáng — không đảo được dấu của bất kỳ phép so sánh nào. Về lý thuyết, BRIEF bất biến hoàn toàn với nhóm biến đổi đó.

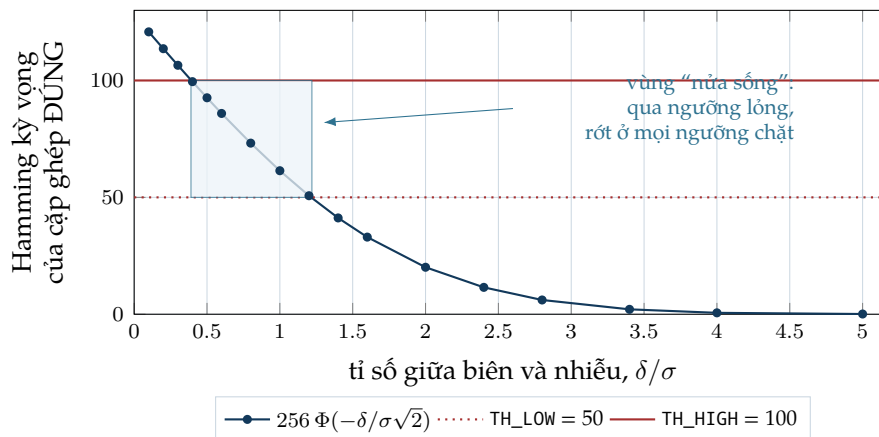
Thứ giết nó là biên. Gọi δ là chênh lệch cường độ thật của một cặp thử, và σ là độ lệch chuẩn của nhiễu trên mỗi mẫu đã làm trơn. Bit lật khi nhiễu đủ lớn để đảo dấu, với xác suất

$$p_{\text{lật}} = \Phi\left(-\frac{\delta}{\sigma\sqrt{2}}\right).$$

Khoảng cách Hamming kỳ vọng giữa hai bộ mô tả của *cùng một* điểm 3D là $256 p_{\text{lật}}$. Đây là đại lượng phải so với hai hằng số mà bạn đã có trong mã: TH_LOW = 50 và TH_HIGH = 100.

Đọc công thức đó theo tỉ số δ/σ thì mọi giá trị tuyệt đối biến mất và cái còn lại mới là thứ dùng được. Khi biên lớn hơn nhiều vài lần, xác suất lật gần như bằng không và khoảng cách Hamming của cặp đúng chỉ vài bit — ghép cặp thoải mái. Khi biên tụt về cùng bậc với nhiễu, Φ bước vào vùng dốc nhất của nó, nên Hamming kỳ vọng nhảy vọt chỉ sau một mức nén tương phản rất nhỏ.

Điều quan trọng không phải nó nhảy bao nhiêu, mà nhảy *vào đâu*. Hai hằng số trong mã chia trực Hamming thành ba vùng, và vùng giữa — trên TH_LOW, dưới TH_HIGH — ứng với một dải δ/σ rất hẹp. Trong dải đó, cặp ghép vẫn qua được mọi hàm dùng ngưỡng lỏng nhưng bị loại ở mọi hàm dùng ngưỡng chặt, tức là tam giác hoá điểm mới và ghép theo túi từ. Hình 1.3 cho thấy toàn bộ đường cong đó; vì trực hoành là một tỉ số không thứ nguyên, hình đúng với mọi mức nhiễu và mọi mức sáng, và bạn chỉ cần biết σ của cảm biến mình để đọc ra ngưỡng δ tương ứng.



Hình 1.3. Vì sao đổi độ sáng giết BRIEF dù BRIEF bất biến với đổi sáng đơn điệu. Đại lượng quyết định là tỉ số giữa biên δ và nhiễu σ , không phải giá trị tuyệt đối của độ sáng. Đường cong là hàm $256 \Phi(-\delta/\sigma\sqrt{2})$, tức là toán chứ không phải số đo; hai đường ngang là hai hằng số có thật trong mã nguồn. Điều đáng mang đi: dải δ/σ nằm giữa hai hằng số ấy rất hẹp, nên chỉ cần tương phản nén đi một chút là cặp ghép rơi từ “qua mọi cửa” xuống “chỉ qua cửa lỏng”. Hệ quả chẩn đoán: khi vào vùng thiếu sáng, thứ tụt trước là số điểm bản đồ mới được tam giác hoá, chứ không phải số cặp bám được.

Điều này giải thích một triệu chứng quen thuộc mà trực giác “đổi sáng làm hỏng mô tả” không giải thích được. Khi robot đi vào hành lang tối, tracking vẫn bám nhưng bản đồ ngừng lớn. Nguyên nhân nằm ở đúng khoảng cách giữa hai ngưỡng trong hình.

TRỰC GIÁC

Coi mỗi bit của BRIEF như một đồng xu có thiên lệch. Thiên lệch đến từ tương phản cục bộ: tương phản càng cao, đồng xu càng chắc chắn rơi về một mặt, và chuỗi 256 mặt đó là vân tay của điểm ảnh. Làm mờ ảnh hay nén tương phản không đổi *mặt* nào được ưa; nó chỉ làm mọi đồng xu công bằng dần. Một vân tay gồm toàn đồng xu công bằng thì khớp với tất cả mọi thứ, tức là không khớp với gì cả. Đó là lý do bộ mô tả không “sai” khi ảnh mờ — nó trở nên vô nghĩa, và hai chuyện này cần hai cách chữa khác nhau.

C. Đổi góc nhìn lớn vượt quá bất biến có sẵn

ORB có đúng hai loại bất biến. Bất biến xoay trong mặt phẳng, nhờ xoay bộ cặp thử theo hướng trọng tâm cường độ. Và bất biến tỉ lệ *đẳng hướng*, nhờ kim tự tháp tám tầng hệ số 1,2. Dải tỉ lệ mà kim tự tháp phủ là $1,2^7 \approx 3,6$ lần.

Đổi góc nhìn ngoài mặt phẳng không nằm trong hai loại đó. Nhìn một mặt phẳng nghiêng một góc θ thì ảnh của nó bị nén theo *một* hướng với hệ số $\cos \theta$, còn hướng vuông góc giữ nguyên. Đó là một biến dạng affine bất đẳng hướng. Kim tự tháp co đều cả hai hướng, nên nó không bù

được. Ở $\theta = 45^\circ$ hệ số nén là 0,71; ở 60° là 0,5. Văn liệu về vùng bất biến affine đã đo điều này từ lâu: điểm số ghép của các bộ mô tả không affine sụp rất nhanh khi góc nhìn ra khỏi vùng gần trục diện, trong khi các vùng bất biến affine giữ được lâu hơn hẳn [4]. Ngưỡng cụ thể phụ thuộc bộ mô tả và bộ dữ liệu, nên nó phải đọc từ chính bài đó chứ không mượn lại ở đây.

Với SLAM cầm tay đi liên tục thì chế độ hồng này hiếm, vì hai khung hình liên tiếp cách nhau vài chục mili-giây. Nó chỉ nổ ở đúng một chỗ: **đóng vòng**. Quay lại một hành lang theo hướng ngược lại là đổi góc nhìn tối đa. Đây là lý do đóng vòng và bám khung hình đòi hai loại đặc trưng khác nhau, và là lý do một hệ dùng chung một bộ mô tả cho cả hai việc luôn phải thỏa hiệp.

↪ **tiền đề** F/29 — chế độ hồng thứ ba là nguyên nhân gốc của phần lớn vòng bị bỏ lỡ; chương 29 mổ xẻ bốn tầng của đóng vòng để xem tầng nào thật sự giết vòng

D. Vùng ít vân: không phải lỗi, mà là hệ quả của thiết kế

Chế độ thứ tư khác hẳn ba chế độ trên. Ba chế độ đầu là chuyện ngưỡng và bất biến, có thể chỉnh. Chế độ thứ tư là hệ quả logic của việc có một bộ phát hiện.

Bộ phát hiện phải ra một quyết định **cứng**, **sớm** và **cục bộ**: điểm ảnh này có đáng giữ không, quyết bằng thông tin trong một cửa sổ bán kính 3. Trên một bức tường trắng, không cửa sổ nào chứa đủ thông tin để trả lời. Hạ ngưỡng xuống 4 thì cái được chọn là nhiều cảm biến, và nhiều không lặp lại giữa hai khung hình. Không có ngưỡng nào đúng, vì thông tin cần thiết không nằm trong cửa sổ đó — nó nằm ở mép tường cách xa hai trăm điểm ảnh.

Đây là điểm cần nhớ khi đọc mục 5. Bộ phát hiện không phải một thủ thuật kỹ thuật; nó là cách duy nhất để hoãn một quyết định mà thông tin cục bộ không đủ để quyết. Cái giá của việc hoãn thì được nói ở đúng mục đó.

E. Bốn chế độ cộng dồn thế nào trong một giây thật

Bốn chế độ ở trên không xảy ra riêng lẻ. Trong một đợt quay nhanh, chúng nối thành một chuỗi, và biết chuỗi đó giúp bạn đọc nhật ký gỡ lỗi nhanh hơn nhiều.

Bước một, tốc độ góc vượt ngưỡng suy ra từ d^* ở mục A, nên tương phản cục bộ tụt và FAST mất phần lớn góc nhỏ. Bước hai, các góc còn sót lại đều là cấu trúc thô, nên chúng định vị kém hơn và sai số tái chiếu của chúng lớn hơn. Bước ba, các góc còn lại phân bố lệch theo hướng vuông góc với chuyển động, nên ma trận thông tin tư thế xấu đi theo đúng một hướng — không phải xấu đều. Bước bốn, số cặp ghép rơi xuống dưới cổng “dưới 15 cặp thì coi như mất bám”, và hệ chuyển sang chế độ dùng IMU dự đoán.

Điểm đáng chú ý là bước năm. Khi hệ quay lại chế độ bám, nó phải ghép khung hình mới với các điểm bản đồ được tạo *trước* đợt mờ. Các điểm đó có bộ mô tả sắc nét, còn khung hình mới vẫn còn hơi mờ, nên khoảng cách Hamming của cặp đúng nằm trong vùng “nửa sống” của Hình 1.3. Kết quả là hệ bám lại chậm hơn mức đáng lẽ có, và trong khoảng chờ đó IMU tích lũy trôi. Phần lớn sai số quỹ đạo do một đợt quay nhanh gây ra không nằm trong lúc quay, mà nằm trong khoảng *sau* khi quay xong.

Chuỗi này có một hệ quả thực hành ngay. Nếu bạn định can thiệp một chỗ duy nhất, hãy can thiệp ở bước năm chứ không ở bước một — nơi ngưỡng ghép cặp *tạm thời* sau một đợt mờ rẻ hơn nhiều so với làm bộ phát hiện chịu mờ giỏi hơn. Đây là suy luận từ cơ chế; mình chưa đo nó, và nó xứng đáng là một thí nghiệm ablation.

Bảng 1.1. Bốn chế độ hỏng, khối bị phá, và thứ thật sự chữa được nó. Cột cuối là cột dùng được nhất: nó cho biết đối bộ đặc trưng có phải là thuốc đúng bệnh không.

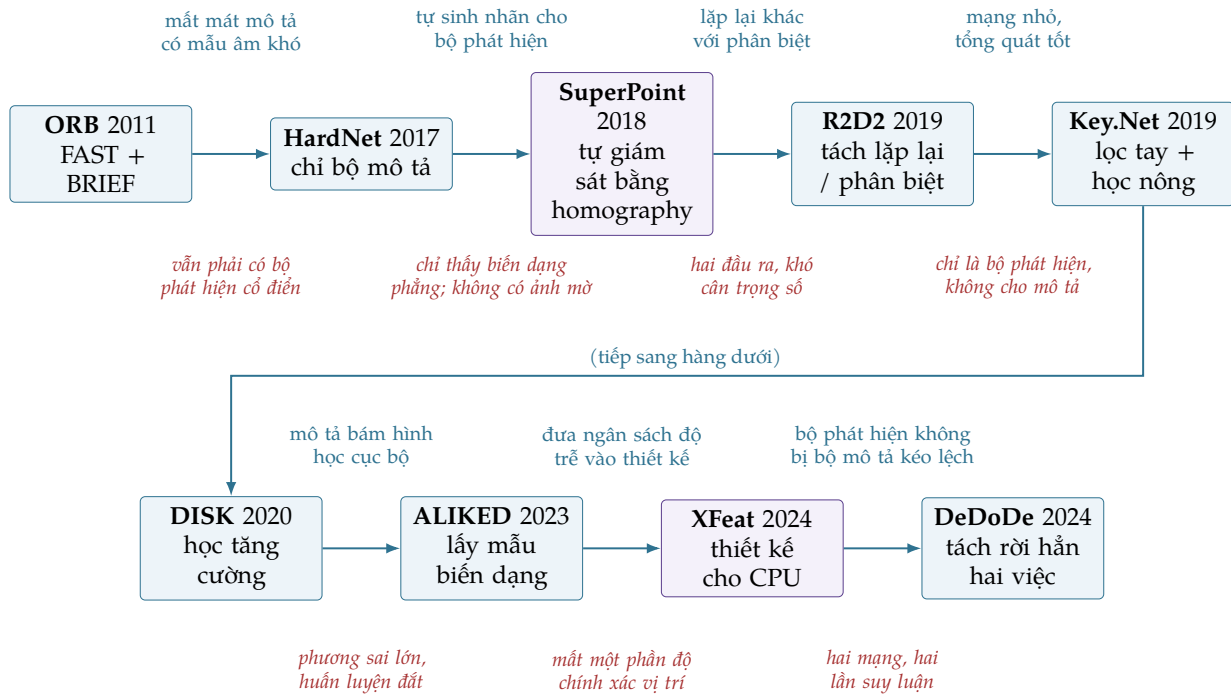
Chế độ	Khối bị phá	Cơ chế	Thuốc đúng bệnh
Ảnh mờ do chuyển động	Bộ phát hiện (FAST)	Trung bình dọc hướng chuyển động kéo tương phản xuống dưới ngưỡng, và cắt đứt cung 9 điểm	Bộ phát hiện học được, hoặc bỏ hẳn bộ phát hiện; giảm phơi sáng nếu đủ sáng; camera sự kiện
Đổi độ sáng	Bộ mô tả (BRIEF)	Biên δ của các phép so sánh co lại, bit lật ngẫu nhiên, Hamming của cặp đúng trôi qua ngưỡng	Bộ mô tả thực học bằng mất mát tương phản (HardNet, SuperPoint)
Đổi góc nhìn lớn	Bộ mô tả và kim tự tháp	Biến dạng affine bất đẳng hướng nằm ngoài bất biến tỉ lệ đẳng hướng	Bộ mô tả học được trên dữ liệu có biến dạng; hoặc bộ ghép cặp có ngữ cảnh toàn cục
Vùng ít vân	Bản thân ý tưởng có bộ phát hiện	Quyết định cứng, sớm, cục bộ trên vùng không có thông tin cục bộ	Ghép cặp dày không cần bộ phát hiện; hoặc chấp nhận và dựa vào IMU

Bảng 1.1 đã trả lời trước một phần câu hỏi của cả chương. Chỉ có hai trong bốn chế độ được chữa bởi việc “đổi ORB sang SuperPoint”. Chế độ thứ nhất được chữa một phần, và chế độ thứ tư thì không — nó cần một loại phương pháp khác hẳn.

1.3 Bộ phát hiện và mô tả học được (Learned Detectors and Descriptors)

Học một bộ mô tả là chuyện dễ hiểu: có nhãn, vì hai ô ảnh của cùng một điểm 3D là một cặp dương. Học một bộ *phát hiện* thì không. Không tồn tại chân trị cho câu “điểm ảnh này có đáng là điểm đặc trưng không”. Toàn bộ lịch sử của dòng này là lịch sử của các cách lách quanh chỗ thiếu nhãn đó, và mỗi cách lách để lại một dấu vết riêng trong cách phương pháp hỏng.

Hình 1.4 xếp cả dòng lên một trục. Mỗi mũi tên ghi nút thắt được gỡ và vấn đề mới sinh ra. Đọc hình trước rồi đọc phần chữ sẽ nhanh hơn hẳn.



Hình 1.4. Dòng đặc trưng học được, đọc theo nút thắt chữ không theo năm. Nhãn phía trên mỗi mũi tên là thứ được gỡ; nhãn phía dưới in nghiêng đỏ là vấn đề mới sinh ra. Hai ô tô tím là hai bước ngoặt về *cách nghĩ*: SuperPoint giải quyết chỗ thiếu nhãn, XFeat là công trình đầu tiên trong dòng đặt ngân sách độ trễ làm ràng buộc thiết kế thay vì làm ghi chú cuối bài.

A. HardNet: hàm mất mát phải mô phỏng đúng quy tắc quyết định lúc chạy

HardNet chỉ làm nửa dễ, tức bộ mô tả, nhưng nó là công trình có bài học thiết kế chuyển được sang việc khác nhiều nhất [5]. Cơ chế của nó gói gọn trong một câu. Lấy một *batch* gồm n cặp ô ảnh khớp nhau, tính ma trận khoảng cách $n \times n$, rồi với mỗi mô neo lấy *mẫu âm gần nhất trong batch* làm mẫu âm, và tối đa hoá biên giữa cặp dương và mẫu âm đó.

Vì sao chọn mẫu âm khó nhất chứ không lấy ngẫu nhiên? Vì lúc chạy thật, đối thủ của một bộ mô tả không phải một ô ảnh ngẫu nhiên. Đối thủ của nó là láng giềng gần nhất trong toàn bộ ảnh kia. Mẫu âm ngẫu nhiên gần như luôn ở rất xa, nên *gradient* tắt và mạng học được rất ít. Sâu hơn nữa: quy tắc quyết định lúc chạy là *phép thử tỉ lệ* giữa ứng viên tốt nhất và ứng viên nhì. Lấy mẫu âm khó nhất trong batch chính là cách xấp xỉ ứng viên nhì đó ngay trong hàm mất mát.

Bài học chuyển được: hàm mất mát nên mô phỏng quy tắc quyết định sẽ dùng lúc chạy, không phải một đại lượng dễ tối ưu hơn. Bạn dùng được bài học này ngay cả khi không dùng mạng nào. Nếu frontend của bạn quyết bằng tỉ lệ giữa hai ứng viên tốt nhất, thì mọi phép đo chất lượng đặc trưng cũng phải đo theo tỉ lệ đó, chứ không đo khoảng cách tuyệt đối.

↔ **cùng nguyên lý với B/09** — đây là dạng cụ thể nhất của nguyên tắc “hàm mất mát mã hoá điều bạn thật sự quan tâm” ở chương thiết kế hàm mất mát

B. SuperPoint: chế ra nhãn từ một phép biến đổi bạn nghịch đảo được

SuperPoint giải chỗ thiếu nhãn bằng ba bước [6]. Bước một, huấn luyện một bộ phát hiện thô trên ảnh tổng hợp gồm các hình khối đơn giản, nơi vị trí góc biết trước theo dựng hình. Bước hai gọi là làm giàu bằng homography: lấy ảnh thật, áp hàng trăm phép homography ngẫu nhiên, chạy bộ phát hiện thô trên từng ảnh đã biến đổi, rồi *ánh xạ ngược* các điểm về ảnh gốc và cộng dồn. Điểm nào sống sót qua nhiều phép biến đổi thì được ghi nhận là điểm giả định. Bước ba, huấn luyện chung một mạng có hai đầu ra — một đầu cho bản đồ điểm, một đầu cho bộ mô tả

256 chiều — với nhân của bộ mô tả lấy từ chính homography đã dùng, vì homography cho biết chính xác điểm nào ứng với điểm nào.

Ý tưởng cốt lõi rất gọn: **khi nhãn không tồn tại, hãy chế nó ra từ một phép biến đổi mà bạn nghịch đảo được chính xác**. Đây là cùng một mạch tư duy với tự giám sát bằng bất biến ở phần C, chỉ khác là phép biến đổi ở đây có ý nghĩa hình học.

Cái giá nằm ở chính chỗ đó, và với người làm SLAM thì đây là điều quan trọng nhất phải biết về SuperPoint. Homography là biến đổi của một *mặt phẳng*. Phân phối huấn luyện vì thế không chứa thị sai do độ sâu, không chứa che khuất, và — điểm chí mạng — **không chứa ảnh mờ do chuyển động** trừ khi bạn tự thêm vào. Nói cách khác, phương pháp được ca ngợi nhiều nhất trong họ này không hề được huấn luyện cho nỗi đau số một của bạn. Nếu định dùng nó cho chuyển động nhanh, việc đầu tiên phải làm là tăng cường dữ liệu bằng mờ có hướng, và đo lại. Đây là một suy luận từ thiết kế của phương pháp, không phải một kết quả đã công bố.

C. R2D2: lặp lại và phân biệt là hai chuyện khác nhau

R2D2 chỉ ra một lỗ hổng khái niệm mà cả ORB lẫn SuperPoint đều mắc [7]. Một điểm có thể rất *lặp lại* mà hoàn toàn vô dụng. Ví dụ chuẩn là góc của một viên gạch trên bức tường gạch. Bộ phát hiện tìm thấy nó ở đúng chỗ trong mọi khung hình, nên tính lặp lại hoàn hảo. Nhưng nó giống hết năm trăm góc gạch khác, nên ghép cặp luôn sai.

Giải pháp là hai đầu ra riêng. Đầu thứ nhất dự đoán tính lặp lại, huấn luyện bằng cách ép bản đồ điểm của ảnh gốc và bản đồ điểm của ảnh đã biến đổi có các đỉnh trùng nhau. Đầu thứ hai dự đoán *độ tin cậy*, tức là ước lượng xem bộ mô tả tại điểm ảnh đó có ghép đúng được không. Điểm được chọn là điểm cao ở cả hai bản đồ.

Bài học ở đây dùng được ngay mà không cần mạng. Bộ chọn điểm của ORB-SLAM3 tối đa hoá đúng một thứ là điểm số góc Harris, tức là một xấp xỉ của tính lặp lại. Nó không có khái niệm nào về tính phân biệt. Đó là nguyên nhân cơ chế của một lỗi rất đắt: vòng đóng sai trên cấu trúc lặp như hàng cột, cửa sổ, hay gạch lát. Nếu bạn muốn một cái tiền rẻ, hãy thêm một điểm số phân biệt cục bộ vào bước chọn điểm — chẳng hạn khoảng cách Hamming tới láng giềng gần nhất *trong cùng ảnh* — và loại các điểm quá giống hàng xóm của chính mình.

D. Key.Net, DISK và DeDoDe: ba cách khác để lách chỗ thiếu nhãn

Ba công trình còn lại tấn công cùng vấn đề bằng ba đường khác nhau, và mỗi đường dạy một điều.

- **Key.Net — dùng lại phần đã giải xong.** Nó đưa các bộ lọc đạo hàm bậc nhất viết tay ở nhiều tỉ lệ vào làm đầu vào, rồi chỉ học một mạng rất nông để tổ hợp chúng [8]. Kết quả là một mô hình rất nhỏ và tổng quát hoá tốt sang ảnh khác kiểu. Bài học: phần nào của bài toán đã có lời giải viết tay tốt thì đừng bắt mạng học lại; hãy để mạng học phần còn lại.
- **DISK — tối ưu đúng mục tiêu rời rạc.** Chọn điểm và chấp nhận một cặp ghép đều là quyết định rời rạc, nên không lấy *gradient* qua được. DISK coi chúng là hành động lấy mẫu từ một phân phối, thưởng +1 cho cặp ghép đúng và phạt -1 cho cặp sai, rồi dùng *gradient* chính sách [9]. Bài học: khi mục tiêu thật là rời rạc và nằm ở cuối đường ống, học tăng cường là lối thoát thay cho việc bịa ra một hàm thay thế khả vi. Cái giá là phương sai *gradient* lớn và huấn luyện đắt hơn hẳn.
- **DeDoDe — tách rời để không kéo lệch nhau.** Huấn luyện chung bộ phát hiện với bộ mô tả có một tác dụng phụ ngầm: bộ phát hiện dần chỉ chọn những điểm mà bộ mô tả *hiện tại* tình cờ mô tả tốt. Hai chế độ hồng quần vào nhau, không tách ra chẩn đoán được. DeDoDe huấn luyện bộ phát hiện riêng, bằng tín hiệu “điểm này có được theo dõi bền qua một tái dựng ba

chiều lớn không”, rồi huấn luyện bộ mô tả riêng [10]. Bài học chính là bài học chẩn đoán, và nó rất hợp với kỷ luật ablation của chương 24.

E. XFeat và ALIKED: hai công trình nói cùng ngôn ngữ với bạn

Hai công trình cuối đáng chú ý riêng, vì chúng là hai công trình duy nhất trong danh sách đặt ràng buộc của bạn vào ngay từ đầu.

XFeat được thiết kế cho CPU và thiết bị biên [11]. Ba lựa chọn thiết kế đáng học. Thứ nhất, mạng rất nông và giảm mẫu sớm, làm việc chủ yếu ở một phần tư độ phân giải. Thứ hai, số kênh ở các tầng đầu bị giữ rất nhỏ, vì các tầng đầu chạy trên độ phân giải cao nên chúng chiếm phần lớn chi phí. Thứ ba, độ chính xác vị trí bị mất do giảm mẫu được lấy lại bằng một mạng tinh chỉnh nhỏ chạy *sau* khi đã ghép cặp, tức là chỉ chạy trên vài trăm điểm thay vì trên toàn ảnh. Đây là mẫu thiết kế đáng nhớ nhất của cả chương: **làm phân thô trên toàn ảnh ở độ phân giải thấp, làm phân tinh chỉ ở nơi cần, sau khi đã biết nơi nào cần**. Cùng mẫu này xuất hiện lại ở LoFTR trong mục 5.

ALIKED tấn công thẳng chế độ hồng thứ ba [12]. Bộ mô tả cổ điển lấy mẫu trên một ô vuông cố định quanh điểm. Khi mặt phẳng nghiêng, ô vuông đó trên ảnh ứng với một hình bình hành trên vật thật, nên nội dung lấy được đã méo trước khi mô tả. ALIKED học các vị trí lấy mẫu *biến dạng được*: mạng dự đoán một tập nhỏ các độ dời, và bộ mô tả lấy mẫu tại các vị trí đã dời đó. Hình dạng vùng hỗ trợ vì thể thích nghi theo hình học cục bộ. Chi phí thấp vì số vị trí lấy mẫu ít.

F. Hai trật tự: phát hiện rồi mô tả, hay mô tả rồi phát hiện

Có một lựa chọn kiến trúc chạy ngầm dưới cả bảy phương pháp trên, và nó đáng được gọi tên riêng. Nó quyết định phương pháp hồng thể nào khi ảnh mờ.

Trật tự cổ điển là **phát hiện rồi mô tả**. FAST chọn điểm trước, BRIEF mô tả sau. SIFT cũng theo đúng trật tự này, chỉ khác ở chỗ nó dùng cực trị của hàm sai khác Gauss thay cho phép thử vòng tròn [13]. LIFT là bản học được đầu tiên của đúng trật tự đó: ba mạng nối tiếp cho phát hiện, ước lượng hướng, và mô tả [14]. Ưu điểm là chi phí thấp, vì bộ mô tả chỉ chạy trên vài trăm ô ảnh. Nhược điểm nằm ở chỗ nếu bộ phát hiện đã bỏ sót một điểm thì bộ mô tả không có cơ hội nào cứu.

Trật tự ngược là **mô tả rồi phát hiện**, do D2-Net đề xuất [15]. Mạng tính một bản đồ đặc trưng dày cho toàn ảnh trước. Sau đó điểm được chọn ngay trên chính bản đồ đặc trưng đó, tại các vị trí vừa là cực đại theo không gian vừa là cực đại theo kênh. Nghĩa là quyết định “đây là điểm đáng giữ” được đưa ra bằng biểu diễn đã qua nhiều tầng tích chập, chứ không bằng cường độ thô.

Khác biệt này quan trọng với bạn vì một lý do rất cụ thể. Ở trật tự cổ điển, ảnh mờ giết bộ phát hiện ở tầng cường độ thô, nơi không có gì cứu được. Ở trật tự ngược, quyết định dựa trên đặc trưng có trường tiếp nhận rộng vài chục điểm ảnh, nên cấu trúc lớn vẫn còn tín hiệu ngay cả khi cấu trúc nhỏ đã bị trung bình mất. Cái giá là chi phí: bản đồ đặc trưng dày phải tính cho toàn ảnh, và độ chính xác vị trí kém hơn vì cực đại được lấy trên lưới đã giảm mẫu. Đây chính là đánh đổi mà XFeat gỡ bằng bước tinh chỉnh sau ghép cặp, và là đánh đổi mà LoFTR đẩy tới cực đoạn ở mục 5.

G. Cần bao nhiêu dữ liệu, và bạn có nó không

Cổng thứ ba của chương 27 hỏi “cần bao nhiêu dữ liệu”. Với đặc trưng học được, câu trả lời dễ chịu hơn hầu hết mọi chỗ khác trong học sâu, và lý do đáng hiểu.

Nhân ở đây không đến từ con người. Nó đến từ hình học. Muốn biết điểm ảnh nào trong ảnh i ứng với điểm ảnh nào trong ảnh j , bạn chỉ cần tư thế tương đối và độ sâu. Vì thế mọi tập dữ liệu huấn luyện của dòng này đều là đầu ra của một hệ thống tái dựng: ảnh thu thập từ internet chạy qua tái dựng cấu trúc từ chuyển động cho cảnh ngoài trời, hoặc ảnh RGB-D quét trong nhà. Không có ai ngồi gán nhãn.

Ba mức đầu tư, theo thứ tự rẻ dần lên đắt dần.

- **Không dữ liệu của bạn.** Dùng trọng số công bố sẵn. Rủi ro là ra ngoài phân phối huấn luyện, và mục “Giới hạn” cuối chương nói rõ kiểu hỏng.
- **Chỉ ảnh của bạn, không cần tư thế.** Đây là chỗ tự giám sát bằng homography có giá trị thực dụng lớn nhất, và nó ít được nói tới. Bạn áp phép biến đổi lên chính ảnh của mình rồi huấn luyện, không cần chân trị nào cả. Nếu miền của bạn khác về *kết cấu* — ảnh hầm mỏ, ảnh ban đêm — thì mức này đã giải quyết phần lớn. Nếu miền của bạn khác về *chuyển động*, hãy thêm mờ có hướng vào tập biến đổi; đó là chỗ SuperPoint để trống.
- **Ảnh của bạn kèm tư thế.** Chạy COLMAP trên các chuỗi đã thu, lấy tư thế và độ sâu thưa, rồi sinh cặp ghép chân trị. Tốn vài ngày tới vài tuần công sức, chủ yếu là công sức lọc các tái dựng hỏng. Đổi lại bạn có một tập huấn luyện đúng miền và đúng chế độ chuyển động của mình.

Nói cách khác, đây là một trong rất ít can thiệp học sâu vào SLAM mà con đường dữ liệu rõ ràng từ đầu tới cuối và không phụ thuộc vào người gán nhãn. Đó là một lý lẽ ủng hộ thật, và nó nên được cân nhắc song song với các lý lẽ phản đối ở mục 6.

Bảng 1.2. Bảy phương pháp, so theo cùng bốn tiêu chí. Cột “hỏng khi nào” là cột quyết định: nó cho biết phương pháp mang theo giả định gì từ dữ liệu huấn luyện của nó. Cột chi phí là xếp hạng tương đối theo số phép nhân cộng và độ sâu mạng, không phải số đo trên Orin.

Tên	Cơ chế	Nguồn giám sát	Hỏng khi nào	Chi phí
HardNet	Chỉ bộ mô tả; triplet với mẫu âm khó nhất trong batch	Cặp ô ảnh khớp từ tái dựng ba chiều	Vẫn cần bộ phát hiện cổ điển, nên thừa hưởng nguyên chế độ hỏng khi ảnh mờ	Rất thấp
SuperPoint	Một mạng, hai đầu ra: bản đồ điểm và mô tả 256 chiều	Tự giám sát bằng homography ngẫu nhiên	Phân phối huấn luyện chỉ có biến dạng phẳng; không có ảnh mờ, không có thị sai	Trung bình
R2D2	Tách hai bản đồ: tính lặp lại và độ tin cậy	Cặp ảnh có dòng quang đã biết	Hai mục tiêu phải cân trọng số bằng tay; nặng hơn SuperPoint	Trung bình cao
Key.Net	Lọc đạo hàm viết tay nhiều tỉ lệ, mạng nông tổ hợp	Mất mát kiểu tính lặp lại trên cặp ảnh	Chỉ là bộ phát hiện; không cho bộ mô tả	Rất thấp
DISK	Chọn điểm và ghép cặp là hành động; thưởng theo cặp ghép đúng	Học tăng cường trên ảnh có tư thế đã biết	Huấn luyện đắt, phương sai lớn, nhảy với cách đặt phần thưởng	Trung bình
XFeat	Mạng rất nông ở một phần tư độ phân giải, tinh chỉnh sau khi ghép	Cặp ảnh tổng hợp và thật	Vị trí kém chính xác hơn ở tầng thô; đổi độ chính xác lấy tốc độ	Thấp
ALIKED	Vị trí lấy mẫu biến dạng được cho vùng hỗ trợ của bộ mô tả	Cặp ảnh có tư thế đã biết	Biến dạng học được vẫn giả định bề mặt trơn cục bộ	Thấp

Mạch 1 — vì sao dòng này tiến theo đúng thứ tự đó

- 1 Vấn đề** Không có chân trị cho câu “điểm nào đáng là điểm đặc trưng”. Mọi cố gắng học bộ phát hiện đều dừng ở đây suốt nhiều năm, trong khi bộ mô tả thì đã học được từ 2016.
- Giải pháp** SuperPoint chế nhạo từ homography nghịch đảo được. Điểm nào sống sót qua hàng trăm phép biến đổi thì được coi là điểm tốt.
- Giả định** Ảnh là của một mặt phẳng, hoặc ít nhất biến dạng giữa hai ảnh xấp xỉ được bằng homography.
- Giá phải trả** Phân phối huấn luyện mất sạch thị sai, che khuất và ảnh mờ. Mạng không bao giờ thấy nỗi đau thật của SLAM.
- Vấn đề mới** Điểm được chọn tối đa hoá tính lặp lại, mà tính lặp lại cao không có nghĩa là ghép đúng được.
- 2 Vấn đề** Cấu trúc lặp cho điểm cực kỳ lặp lại nhưng không phân biệt được, nên ghép cặp sai một cách có hệ thống.
- Giải pháp** R2D2 dự đoán hai bản đồ tách rời, chọn điểm cao ở cả hai.
- Giá phải trả** Hai hàm mất mát phải cân trọng số bằng tay, và trọng số đó không chuyển được sang tập dữ liệu khác.
- Vấn đề mới** Bộ phát hiện và bộ mô tả vẫn huấn luyện chung, nên bộ phát hiện bị kéo về phía sở thích của bộ mô tả.
- 3 Vấn đề** Mọi hàm mất mát ở trên đều là hàm thay thế. Thứ ta thật sự muốn — số cặp ghép đúng sau bước lọc — là một đại lượng rời rạc.
- Giải pháp** DISK tối ưu thẳng đại lượng rời rạc đó bằng *gradient* chính sách; DeDoDe tách hẳn hai việc để chúng không kéo lệch nhau.
- Giá phải trả** Huấn luyện đắt hơn nhiều và nhạy với cách đặt phần thưởng; DeDoDe phải chạy hai mạng.
- Hệ quả** Tới đây chất lượng đã bão hoà trên các bảng xếp hạng ghép ảnh, và trực cạnh tranh chuyển sang *chi phí*. Đó chính là chỗ XFeat và ALIKED bước vào, và cũng là chỗ người làm SLAM nên bắt đầu đọc.

1.4 Ghép cặp học được và chi phí bậc hai (Learned Matching)

Ghép cặp trong ORB-SLAM3 là một chuỗi các quyết định *độc lập và cục bộ*. Với mỗi bộ mô tả, tìm láng giềng gần nhất theo Hamming. Loại nếu tỉ lệ giữa ứng viên nhất và nhì lớn hơn mfNNratio . Giữ nếu hai chiều cùng chọn nhau. Cuối cùng, bỏ phiếu bằng biểu đồ chênh lệch góc với HISTO_LENGTH bằng 30, và giữ ba ô cao nhất.

Đọc lại danh sách đó theo con mắt khác thì nó là một xấp xỉ thô của hai ràng buộc toàn cục. Phép thử tỉ lệ là xấp xỉ cục bộ của ràng buộc *mỗi điểm chỉ được dùng một lần*. Biểu đồ góc là xấp xỉ rất thô của ràng buộc *mọi cặp ghép phải nhất quán với một phép biến đổi chung*. SuperGlue là câu trả lời cho câu hỏi: nếu ta thay hai xấp xỉ thô đó bằng hai thứ thật thì được gì.

A. SuperGlue: đưa ngữ cảnh và ràng buộc một-một vào trong mô hình

SuperGlue nhận vào hai tập điểm kèm bộ mô tả và trả về ma trận gán [16]. Bốn bước, mỗi bước gỡ một thứ.

- Nhúng vị trí.** Mỗi điểm thành một vector, ghép từ bộ mô tả và từ một phép nhúng của toạ độ cùng điểm số tin cậy. Từ lúc này, “ở đâu” và “trông thế nào” nằm chung một biểu diễn.

2. **Chín tầng *attention* xen kẽ.** Tầng tự chú ý cho mỗi điểm nhìn mọi điểm khác trong *cùng* ảnh; tầng chú ý chéo cho nó nhìn mọi điểm trong ảnh kia. Sau chín tầng, biểu diễn của một điểm không còn là mô tả cục bộ nữa mà đã mang thông tin của cả hai ảnh.
3. **Ma trận điểm số kèm thùng rác.** Tích vô hướng giữa hai tập vectơ cho ma trận $N \times N$, thêm một hàng và một cột “thùng rác” để chứa các điểm không có cặp.
4. **Sinkhorn.** Chuẩn hoá xen kẽ theo hàng và theo cột nhiều vòng, tức là giải xấp xỉ bài toán vận tải tối ưu có chính quy hoá entropy [17]. Kết quả là ma trận gần như một-một, và tính một-một do *mô hình* bảo đảm chứ không do một phép lọc sau đó.

Vì sao cơ chế này thắng phép thử tỉ lệ? Vì phép thử tỉ lệ chỉ nhìn được hai ứng viên của riêng một điểm. Trên một bức tường gạch, cả hai ứng viên đều giống nhau, nên phép thử loại luôn cặp ghép — kể cả khi cặp đúng nằm trong đó. Chú ý chéo thì cho điểm đó biết rằng các hàng xóm của nó đều ghép về một vùng nhất quán, và thông tin đó đủ để chọn đúng viên gạch. Nói gọn: **SuperGlue đổi một quyết định cục bộ lấy một quyết định toàn cục, và trả bằng chi phí bậc hai.**

B. Chi phí bậc hai, và vì sao nó chi phối mọi quyết định khác

Chi phí của *attention* tỉ lệ với bình phương số điểm. Kế toán thô nhưng đủ dùng: mỗi phép chú ý trên N điểm với số chiều d tốn khoảng $2N^2d$ phép nhân cộng, gồm ma trận điểm số và tổng có trọng số. SuperGlue có chín tầng, mỗi tầng áp cho cả hai ảnh, nên khoảng 18 phép chú ý:

$$\text{MAC} \approx 18 \cdot 2N^2d = 36N^2d.$$

Công thức đó là toàn bộ thứ bạn cần, và nó nói hai điều. Thứ nhất, chi phí tăng theo N^2 : nhân đôi số điểm là gấp bốn chi phí, nên số điểm là đòn bẩy mạnh nhất mà bạn có, mạnh hơn mọi thủ thuật cài đặt. Thứ hai, hệ số đứng trước N^2 không nhỏ — chín tầng, mỗi tầng áp cho cả hai ảnh, số chiều d cỡ vài trăm — nên ngay ở số điểm mà ORB-SLAM3 đang dùng, đây đã là một khối lượng tính toán khác hẳn *bậc* so với một phép tìm láng giềng gần nhất trên chuỗi bit. Khác bậc, không phải khác vài phần trăm.

Đổi từ phép nhân cộng sang mili-giây thì chương này dừng lại, và dừng có lý do. Muốn chia thì phải có mẫu số, tức thông lượng thật của Orin ở batch bằng 1 trên đúng dạng phép toán này; mà *attention* với ma trận nhỏ thường bị chặn bởi băng thông bộ nhớ chứ không bởi số nhân, nên mẫu số ấy cách hiệu năng đỉnh rất xa và không tra được ở đâu cả. Một con số mili-giây suy ra từ một mẫu số đoán mò là con số vô giá trị — tệ hơn cả không có, vì nó sẽ được nhớ và được trích lại — nên ở đây không có con số nào. Cách đo mẫu số đó nằm ở mục 6.

Thứ chuyển được sang mọi phần cứng là *độ dốc*: mọi thứ ở đây tăng theo N^2 , nên quyết định về số điểm quan trọng hơn mọi thứ khác, và nó vẫn đúng nguyên khi bạn đổi máy.

C. LightGlue: ba cách rút ngắn cùng một chồng tầng

LightGlue giữ nguyên ý tưởng của SuperGlue và tấn công riêng phần chi phí [18]. Ba cơ chế, và cả ba đều dùng lại được ở nơi khác.

- **Thoát sớm thích nghi.** Sau mỗi tầng, một bộ phân lớp nhỏ ước lượng xem lời giải đã đủ chắc chưa. Cặp ảnh dễ — và trong SLAM thì gần như mọi cặp khung hình liên tiếp đều dễ — dừng sau vài tầng thay vì chạy hết chín tầng. Ý tưởng gốc: *độ sâu tính toán nên tỉ lệ với độ khó của mẫu, không phải là hằng số.*
- **Cắt điểm.** Điểm nào được dự đoán là không thể ghép thì bị loại giữa chừng, nên các tầng sau chạy trên N nhỏ hơn. Vì chi phí là bậc hai, bỏ một nửa số điểm làm phần còn lại rẻ đi bốn lần. Đây là chỗ tiết kiệm lớn nhất.

- **Bỏ Sinkhorn.** Thay vì hàng chục vòng chuẩn hoá, LightGlue dự đoán riêng một điểm số “có ghép được không” cho từng điểm, rồi kết hợp nó với ma trận tương đồng trong một bước. Phần gán không còn là vòng lặp nữa.

Bài báo báo cáo tăng tốc vài lần so với SuperGlue ở cùng mức chính xác, và mức tăng lớn hơn trên cặp ảnh dễ. Mình không dẫn con số mili-giây cụ thể ở đây, vì con số đó gần với phần cứng máy để bàn trong bài và không chuyển sang Orin được.

D. Ba thứ lấy được từ SuperGlue mà không cần chạy SuperGlue

Trước khi bàn chi phí, đáng dừng lại ở một câu hỏi rõ hơn nhiều: trong ý tưởng của SuperGlue, phần nào dùng được ngay trong ORBmatcher hiện tại, không cần mạng nào?

- **Gán một-một thật thay cho phép thử tỉ lệ.** Bạn đã có ma trận chi phí Hamming giữa các ứng viên. Giải bài toán gán trên ma trận đó — bằng thuật toán Hungary, hoặc bằng vài vòng chuẩn hoá kiểu Sinkhorn — cho ra một lời giải toàn cục thay vì một chuỗi quyết định độc lập. Chi phí là bậc ba theo số ứng viên trong cửa sổ tìm kiếm, mà số đó rất nhỏ, nên trên thực tế nó gần như miễn phí. Đây là phần rẻ nhất và mình cho là đáng thử trước.
- **Bỏ phiếu nhất quán mịn hơn.** Biểu đồ chênh lệch góc với 30 ô là một phép bỏ phiếu một chiều rất thô. Bỏ phiếu trên một không gian hai chiều — chênh lệch góc cộng với độ dời trung bình — bắt được nhiều cặp sai hơn với cùng chi phí. Đây đúng là thứ mà tầng *attention* của SuperGlue học được, chỉ ở dạng viết tay.
- **Điểm số “có ghép được không” riêng cho từng điểm.** LightGlue dự đoán nó bằng mạng. Bạn xấp xỉ được nó bằng một đại lượng có sẵn: khoảng cách Hamming tới láng giềng gần nhất *trong cùng ảnh*. Điểm nào có láng giềng cùng ảnh rất gần thì nằm trên cấu trúc lặp, và nên bị hạ trọng số. Đây cũng chính là ý tưởng “tính phân biệt” của R2D2, áp bằng tay.

Ba việc này không thay thế được SuperGlue. Nhưng chúng cho bạn một *baseline* mạnh hơn trước khi so, và mục 6 sẽ giải thích vì sao có một *baseline* mạnh lại quan trọng tới vậy ở đây.

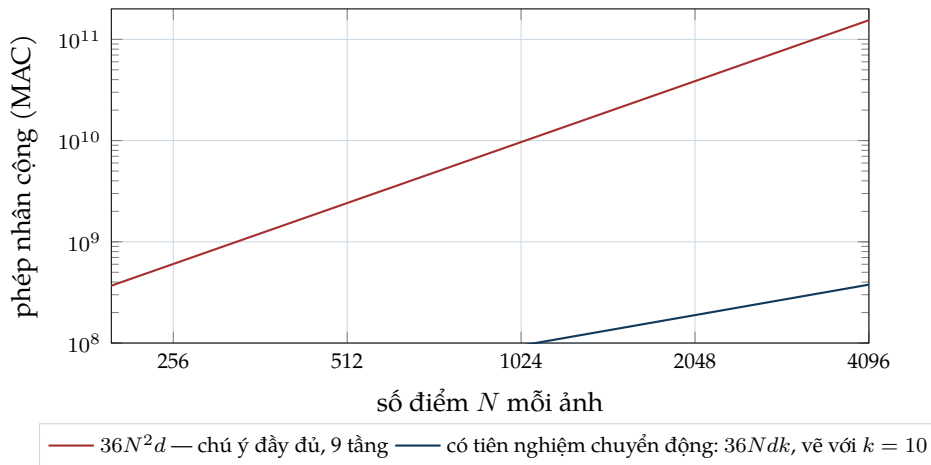
E. Sáu cách né chi phí bậc hai, xếp theo mức hữu ích cho SLAM

Đây là mục đáng giá nhất của cả phần ghép cặp, vì nó là chỗ nền SLAM của bạn có lợi thế thật so với người viết các bài báo trên.

1. **Giảm N .** Rẻ nhất và hiệu quả nhất, vì tiết kiệm là bậc hai: giảm một nửa số điểm thì chi phí còn một phần tư. Hãy đo APE theo số điểm trước khi làm bất cứ việc gì khác — rất có thể hệ của bạn đã bão hoà ở một số điểm thấp hơn hẳn cấu hình mặc định, và nếu đúng thế thì bạn vừa lấy được phần lớn khoản tiết kiệm mà không đụng tới một mô hình nào.
2. **Chỉ chạy ở nơi thật sự cần.** Giữ ORB cho bám khung hình ở 30 Hz, và chỉ gọi bộ ghép cặp học được cho ứng viên đóng vòng và định vị lại, ở cỡ 1 Hz. Ngân sách giảm khoảng ba mươi lần, và nó nhắm đúng chế độ hồng thứ ba — chế độ mà đối góc nhìn lớn thật sự xảy ra. Theo mình đây là phương án có tỉ lệ lợi ích trên rủi ro cao nhất trong cả chương.
3. **Dùng dự đoán chuyển động để giới hạn cặp ứng viên.** Hệ của bạn đã có mô hình vận tốc không đổi và có IMU. Chúng cho một cửa sổ tìm kiếm hẹp quanh vị trí chiếu. Khi đó mỗi điểm chỉ còn k ứng viên trong cửa sổ thay vì cả N điểm của ảnh kia, nên chi phí thành $N \cdot k$ chứ không phải N^2 , tức là gần như tuyến tính — và k là một đại lượng bạn đo được ngay từ nhật ký của ORBmatcher. Về mặt *attention*, đây chính là chú ý thừa theo khối.
4. **Cắt điểm và thoát sớm.** Đúng hai cơ chế của LightGlue; dùng được ngay vì đã có mã nguồn mở.

5. **Chú ý tuyến tính.** Thay hàm mũ trong *softmax* bằng một ánh xạ đặc trưng để đổi thứ tự phép nhân ma trận, cho chi phí $O(N)$ [19]. LoFTR dùng cách này. Cái giá là sức biểu diễn giảm, và với N nhỏ thì nó còn chậm hơn vì hằng số lớn.
6. **Ghép thô rồi tinh.** Ghép ở độ phân giải thấp trước rồi chỉ tinh chỉnh quanh các cặp đã thắng. Đây là mẫu thiết kế đã gặp ở XFeat và sẽ gặp lại ở LoFTR.

Điểm số ba đáng được nói thẳng, vì nó là một khoảng trống thật trong văn liệu. Gần như mọi bài báo ghép cặp đều đánh giá trên cặp ảnh hai khung nhìn rộng, lấy từ tái dựng ba chiều, *không có tiên nghiệm chuyển động nào*. Trong bối cảnh đó, giải bài toán gán toàn cục với chi phí N^2 là hợp lý. Trong bối cảnh của bạn, giữa hai khung hình cách nhau 33 mili-giây và có IMU nối chúng, phần lớn chi phí N^2 đó là tiền trả cho một bài toán bạn không có. Hình 1.5 vẽ khoảng cách giữa hai chế độ đó.



Hình 1.5. Chi phí ghép cặp theo số điểm. Hai đường là hai công thức được vẽ ra chứ không phải hai phép đo: đỏ là chú ý đầy đủ, xanh là cùng mô hình nhưng chỉ xét k ứng viên nằm trong cửa sổ do IMU dự đoán. Khoảng cách giữa hai đường đúng bằng N/k , nên nó rộng ra khi bạn xin nhiều điểm hơn và hẹp lại khi tiên nghiệm của bạn kém đi — đó là toàn bộ nội dung của hình. Trục tung đếm phép nhân cộng chứ không đếm mili-giây; muốn đổi sang thời gian thì phải có thông lượng đo được của máy bạn ở batch bằng 1, và mục 6 nói cách lấy nó.

CẠM BẮY

Bảng xếp hạng ghép cặp đo trên cặp ảnh hai khung nhìn rộng, chọn ra từ tái dựng ba chiều, thường cách nhau nhiều mét và nhiều chục độ [20]. Frontend của bạn ghép hai khung hình cách nhau 33 mili-giây, với một tiên nghiệm tư thế tốt tới vài điểm ảnh. Hai bài toán này khác nhau tới mức thứ hạng trên bảng kia gần như không dự đoán được gì cho bạn. Chuyển kết luận thẳng từ bảng xếp hạng sang hệ của mình là sai lầm đắt nhất trong cả chương — không phải vì các con số sai, mà vì chúng trả lời một câu hỏi khác. Cách kiểm tra rẻ: đo phân bố *góc giữa hai tia quan sát* của các cặp ghép trong chuỗi của bạn, rồi so với phân bố đó trong tập kiểm thử của bài báo.

1.5 Bỏ hẳn bộ phát hiện (Detector-free Matching)

Mục 2 đã kết luận rằng vùng ít vân không phải một lỗi cài đặt. Nó là hệ quả logic của việc bắt một cửa sổ bán kính 3 quyết định một câu hỏi mà thông tin trả lời nằm cách đó hai trăm điểm ảnh. Có đúng một cách thoát: đừng quyết sớm.

A. LoFTR: ghép dày ở tầng thô, rồi tinh chỉnh

LoFTR bỏ hẳn bước chọn điểm [21]. Nó chạy hai giai đoạn.

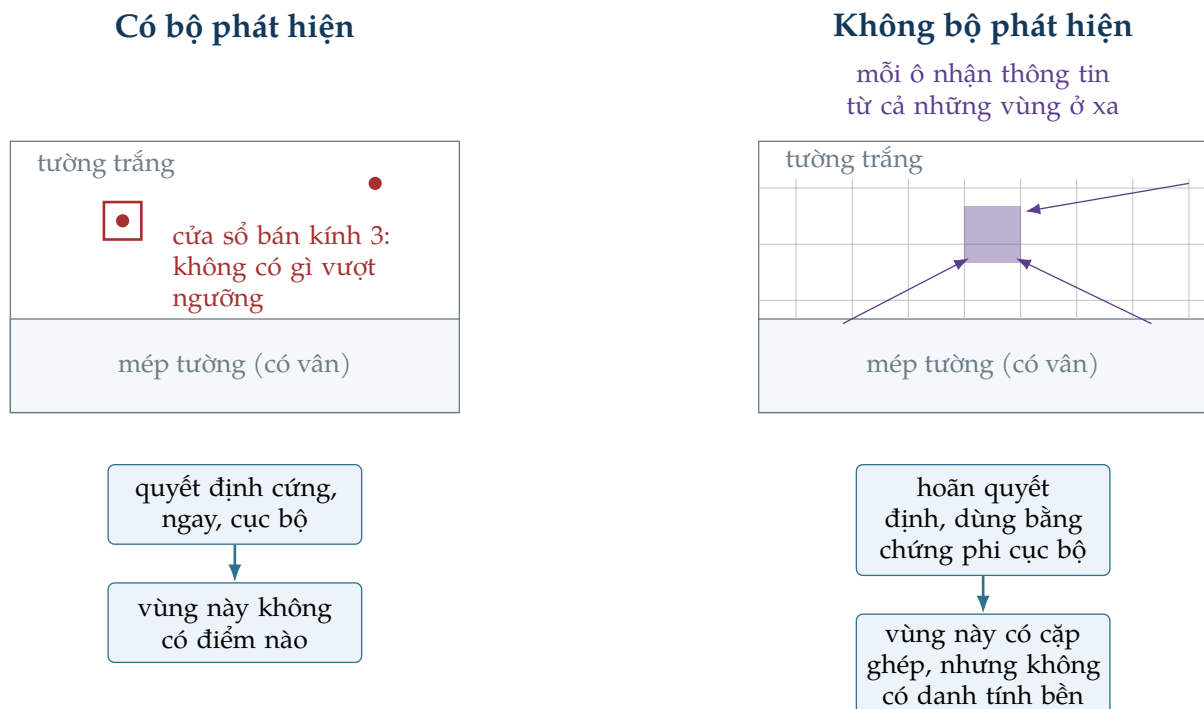
Giai đoạn thô làm việc trên bản đồ đặc trưng ở một phần tám độ phân giải. Với ảnh 640 nhân 480, đó là lưới 80 nhân 60, tức 4800 ô. Mỗi ô thành một *token*, rồi một chồng tầng tự chú ý và chú ý chéo cho mỗi ô nhìn thấy toàn bộ hai ảnh. Vì 4800 ô mà chú ý đầy đủ thì ma trận điểm số có hơn hai chục triệu ô, LoFTR bắt buộc phải dùng chú ý tuyến tính. Sau đó một tầng ghép khả vi — *softmax* hai chiều hoặc vận tải tối ưu — cho ra tập cặp ghép ở mức lưới thô.

Giai đoạn tinh chạy trên từng cặp đã thắng. Nó cắt một cửa sổ nhỏ quanh mỗi cặp trên bản đồ đặc trưng ở một phần hai độ phân giải, tương quan hai cửa sổ, rồi lấy kỳ vọng trên bản đồ tương quan để có vị trí dưới mức điểm ảnh. Đây đúng là mẫu thiết kế đã gặp ở XFeat: thô trên toàn ảnh ở độ phân giải thấp, tinh chỉ ở nơi đã biết là cần.

B. Vì sao bỏ bộ phát hiện lại cứu được vùng ít vân

Câu trả lời hay bị nói sai. Bộ phát hiện *không* làm bức tường trắng có vân. Nó đối thứ khác: *đổi loại bằng chứng* được phép dùng để quyết định một cặp ghép.

Bộ phát hiện dùng bằng chứng cục bộ và quyết ngay. Trên tường trắng thì bằng chứng cục bộ bằng không, nên quyết định nào cũng sai. Chồng tầng chú ý của LoFTR làm cho biểu diễn của một ô ở giữa bức tường mang thông tin từ mép tường, từ khung cửa, từ bố cục cả căn phòng. Ô đó vẫn “không có vân”, nhưng cặp ghép của nó được quyết bằng bằng chứng *phi cục bộ*. Hình 1.6 vẽ đúng khác biệt đó.



Hình 1.6. Vì sao bỏ bộ phát hiện cứu được vùng ít vân. Bên trái, cửa sổ bán kính 3 ở giữa bức tường không chứa thông tin nào, nên mọi ngưỡng đều sai. Bên phải, chồng tầng chú ý đưa thông tin từ mép tường và từ bố cục phòng vào chính ô đó, nên cặp ghép được quyết bằng bằng chứng phi cục bộ. Cái mất nằm ở ô cuối bên phải: cặp ghép có, nhưng điểm đó không mang một danh tính có thể tìm lại ở khung hình sau — và danh tính mới là thứ SLAM cần để dựng bản đồ.

C. Bốn cái giá, và cái thứ tư mới là cái giết

- **Chi phí sần theo diện tích ảnh, không theo số điểm.** Bạn không giảm được chi phí bằng cách xin ít điểm hơn. Sần được đặt bởi độ phân giải, và nó cao.
- **Cặp ghép neo vào lưới.** Bước tinh chỉnh lấy lại độ chính xác vị trí, nhưng không lấy lại được tính lặp lại của *vị trí trên vật thật* qua nhiều khung hình.
- **Chỉ hai ảnh một lúc.** LoFTR ghép ảnh với ảnh. SLAM cần một điểm bản đồ được quan sát bởi nhiều khung khoá, tức là một vệt theo dõi. Nối các cặp ghép hai-ảnh thành vệt thì tích lũy sai lệch và đứt ngay khi bỏ qua một khung.
- **Không có bộ mô tả để lưu.** Đây là cái giá quyết định. Bám khung hình là bài toán một-nhiều: ghép khung mới với hàng vạn điểm bản đồ. Có bộ mô tả thì đó là một phép tìm láng giềng gần nhất. Không có bộ mô tả thì bạn phải chạy mạng *một lần cho mỗi cặp ảnh ứng viên*. Với N khung khoá là N lần chạy mạng. Và túi từ nhị phân [22] — nền của cả định vị lại lẫn lấy ứng viên vòng — không còn dùng được, vì nó cần bộ mô tả.

D. RoMa và nhánh ghép dày mạnh nhất

RoMa đẩy ý tưởng đi xa hơn: thay vì một tập cặp ghép, nó hồi quy một trường biến dạng dày cho toàn ảnh, kèm một bản đồ độ chắc chắn theo từng điểm ảnh [23]. Đặc trưng thô lấy từ một mô hình nền thị giác đông lạnh, đặc trưng tinh lấy từ một mạng tích chập chuyên dụng. Điểm mạnh nhất của nó là các cặp ảnh cực khó: đổi góc nhìn rất lớn, đổi mùa, đổi ánh sáng. Đây là loại cặp ảnh mà mọi phương pháp thừa đều bỏ cuộc.

Cái giá tương ứng. Xương sống là một *transformer* lớn, nên chi phí ở bậc hoàn toàn khác so với XFeat hay LightGlue. Với ngân sách một khung hình trên Orin thì nhánh này nằm ngoài tầm, và mình không thấy lý do kỹ thuật nào để nghĩ khác. Có một bản kế nhiệm là RoMa v2, công bố năm 2025, được giới thiệu là nhanh hơn và dày hơn; mình chưa đọc kỹ nên chỉ nhắc tên và không dẫn bất kỳ con số nào. Ở nhánh nhẹ hơn, Efficient LoFTR kéo chi phí của ghép bán dày về gần mức của bộ ghép thưa bằng cách gộp phép chú ý và làm gọn giai đoạn thô [24].

E. Nếu vẫn muốn dùng ghép dày trong SLAM thì phải thêm gì

Giả sử bạn quyết định dùng LoFTR ở tầng đóng vòng. Có ba mảnh còn thiếu, và biết trước ba mảnh đó giúp bạn ước lượng đúng khối lượng công việc.

- **Một tầng lấy ứng viên.** Ghép dày chỉ trả lời câu “hai ảnh này khớp tới đâu”. Nó không trả lời câu “ảnh nào trong một vạn khung khoá là ứng viên”. Bạn vẫn cần bộ mô tả toàn ảnh cho bước đó, và đó là nội dung chương 29.
- **Một cách gán danh tính.** Sau khi có cặp ghép giữa khung mới và khung khoá, bạn phải nối chúng vào các điểm bản đồ đã tồn tại. Cách rẻ là chiếu ngược: điểm ảnh được ghép nằm gần điểm đặc trưng ORB nào của khung khoá thì nhận danh tính của điểm bản đồ đó. Cách này hoạt động, và nó cho thấy hai họ bổ sung nhau chứ không loại trừ nhau.
- **Một mô hình bất định.** Cặp ghép của LoFTR không có tầng kim tự tháp, nên không có `mvInvLevelSigma2`. Bạn phải lấy trọng số từ chỗ khác — điểm số tin cậy của tầng ghép, hoặc độ nhọn của bản đồ tương quan ở bước tinh chỉnh. Đây lại là đúng vấn đề của mục 6E, xuất hiện lần thứ hai, và sự lặp lại đó là dấu hiệu nó là vấn đề trung tâm chứ không phải chi tiết.

Bảng 1.3. Ba họ ghép cặp, so theo cùng tiêu chí, kèm cột quan trọng nhất với bạn: nên đặt nó ở đâu trong hệ SLAM. Cột chi phí là bậc tương đối theo số phép nhân cộng, không phải số đo.

Họ	Cơ chế	Mạnh khi nào	Hỏng khi nào	Đặt ở đâu
Thừa cổ điển (ORB)	Láng giềng gần nhất theo Hamming, phép thử tỉ lệ, biểu đồ góc	Khung nhìn hẹp, có tiên nghiệm tốt, cần độ trễ thấp và đoán trước được	Ảnh mờ, đối sáng mạnh, đối góc nhìn lớn, vùng ít vân	Bám khung hình ở 30 Hz
Thừa học được (SuperPoint + LightGlue)	Đặc trưng học được cộng gán toàn cục bằng <i>attention</i>	Khung nhìn rộng, cấu trúc lặp, đối sáng; vẫn cho bộ mô tả lưu lại được	Ra ngoài phân phối huấn luyện; chi phí bậc hai theo số điểm	Kiểm chứng ứng viên đóng vòng, định vị lại
Dây không bộ phát hiện (LoFTR, RoMa)	Ghép mọi ô lưới, không chọn điểm; thô rời tinh	Vùng ít vân, đối góc nhìn cực lớn, cặp ảnh khó nhất	Không có bộ mô tả để lưu; chỉ hai ảnh một lúc; chi phí theo diện tích ảnh	Ghép ngoại tuyến, tái dựng, hoặc kiểm chứng vòng khi có ngân sách

Kết luận của mục này trùng với kết luận của mục 4, và sự trùng đó đáng chú ý. Cả hai nhánh — ghép cặp học được và ghép dây không bộ phát hiện — đều quá đắt cho mỗi khung hình, và đều mạnh nhất đúng ở chỗ không có tiên nghiệm chuyển động. Chỉ có một chỗ trong hệ SLAM thoả cả hai điều kiện đó: **kiểm chứng ứng viên đóng vòng và định vị lại**. Đó là nơi nên đặt chúng.

→ **hệ quả của** F/29 — chương 29 nhận đúng kết luận này làm điểm xuất phát và đo xem tầng nào của đóng vòng thật sự là nút thắt

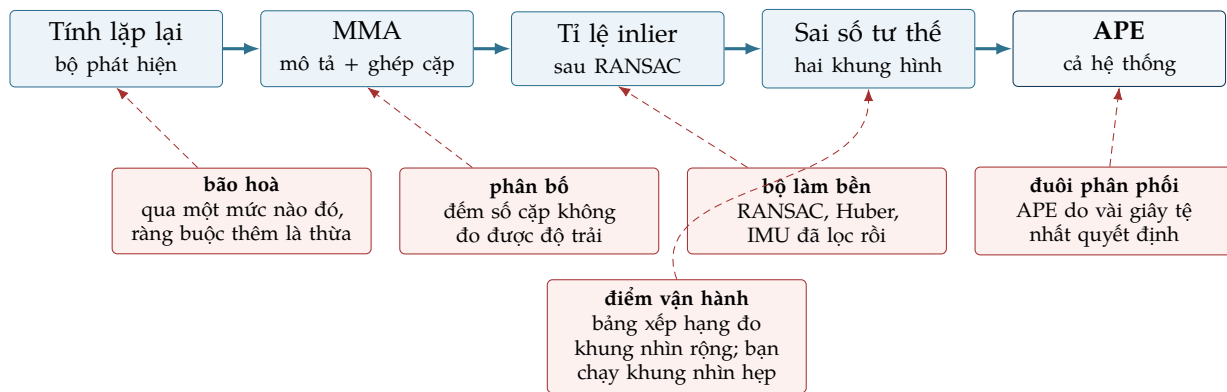
1.6 Đánh giá cho hệ thời gian thực (Evaluation for a Real-time System)

Đây là mục quan trọng nhất của chương. Ba mục trước là bản đồ; mục này là cái cân.

A. Bốn thước đo, mỗi cái cô lập một khối

- **Tính lặp lại** — tỉ lệ điểm được tìm thấy ở cùng vị trí trên vật thật trong hai ảnh. Nó cô lập riêng *bộ phát hiện*.
- **Độ chính xác ghép trung bình (MMA)** — trong các cặp được ghép, tỉ lệ cặp sai dưới ngưỡng vài điểm ảnh. Nó đo *bộ mô tả cộng bộ ghép cặp*, thường trên HPatches [25].
- **Tỉ lệ inlier sau RANSAC** — mức sạch của tập đưa vào tối ưu. Nó đo *toàn bộ frontend cộng mô hình hình học*.
- **APE** — con số duy nhất mà hệ của bạn bị đánh giá.

Điểm mấu chốt của cả chương nằm ở đây: **ba thước đo đầu có thể tăng mạnh mà APE không đổi một milimét**. Đây không phải nghịch lý, và nó có đúng năm cơ chế. Hình 1.7 vẽ chỗ tín hiệu bị hấp thụ ở từng nấc.



Hình 1.7. Thang đo từ frontend tới APE, và năm chỗ tín hiệu bị hấp thụ trên đường đi. Mỗi ô đỏ là một cơ chế độc lập; chúng cộng dồn, nên một cải thiện lớn ở nấc trái có thể tới nấc phải mà không còn gì. Hệ quả thực hành: đừng báo cáo cải tiến ở ba nấc trái rồi ngụ ý nấc phải. Hãy đo nấc phải, hoặc đo trần của nó bằng *oracle*.

B. Năm cơ chế hấp thụ, nói rõ từng cái

1. **Bảo hoà.** Tư thế có sáu bậc tự do, và mỗi cặp ghép cho hai phương trình. Vài chục cặp trải đều đã thừa sức xác định sáu ẩn đó; vượt qua mức bảo hoà — mức mà bạn đo được bằng cách quét số điểm và nhìn APE — mỗi cặp ghép thêm chỉ làm ma trận thông tin dày lên theo những hướng vốn đã chắc. Ràng buộc chặt chuyển sang chỗ khác: đường cơ sở, phân bố độ sâu, và trọng số IMU.
2. **Phân bố quan trọng hơn số lượng.** Rất nhiều cặp ghép dồn vào một góc ảnh cho ma trận thông tin điều kiện xấu hơn hẳn một số cặp ít hơn nhiều nhưng trải đều khắp khung hình. Không thước đo nào ở trên đo độ trải. Nếu muốn một chỉ số tốt hơn, hãy đo trị riêng nhỏ nhất của ma trận thông tin tư thế, không phải đếm cặp ghép.
3. **Bộ làm bền đã ăn phần lớn lợi ích.** Hệ của bạn có RANSAC, mô hình vận tốc không đổi, hàm mất mát Huber trong tối ưu, và ràng buộc IMU. Bốn thứ đó đã loại phần lớn cặp ghép xấu. Làm sạch đầu vào chỉ khiến chúng đỡ vất vả hơn. Đầu vào tốt hơn chỉ tạo khác biệt ở đúng nơi bộ làm bền đã *thất bại*, mà nơi đó chiếm rất ít thời lượng.
4. **Đuôi phân phối quyết định APE.** APE bị đặt bởi vài giây tệ nhất của chuỗi. Ba thước đo kia là trung bình trên toàn tập kiểm thử. Một cải thiện trung bình lớn hoàn toàn tương thích với việc vài giây tệ nhất không đổi gì.
5. **Sai điểm vận hành.** Bảng xếp hạng đo cặp ảnh khung nhìn rộng, còn bạn ghép hai khung hình cách nhau 33 mili-giây. Ở khung nhìn hẹp có tiên nghiệm tốt, ORB đã gần tối ưu, nên khoảng trống để cải thiện vốn đã nhỏ.

C. Thí nghiệm quyết định: *oracle* ghép cặp

Từ năm cơ chế trên suy ra một quy trình rất gọn, và nó nên chạy *trước* khi tích hợp bất cứ mạng nào.

Thay đầu ra của bộ ghép cặp bằng cặp ghép dựng từ chân trị — chiều điểm bản đồ bằng tư thế chân trị và độ sâu chân trị, rồi lấy cặp ghép trong bán kính nhỏ. Chạy lại toàn hệ. APE thu được là **trần** mà mọi cải tiến frontend bị chặn trên. Nếu trần đó chỉ tốt hơn APE hiện tại vài phần trăm, bạn vừa tiết kiệm được ba tháng. Nếu nó tốt hơn hẳn, bạn vừa biết đầu tư vào đâu và biết trước lợi ích tối đa.

Chạy thêm hai biến thể sẽ tách được câu hỏi ra rõ hơn nữa. Biến thể một: chỉ bật *oracle* trong các đoạn ảnh mờ, để đo riêng phần đóng góp của chế độ hồng thứ nhất. Biến thể hai: giữ nguyên cặp ghép của ORB nhưng thay *trọng số* của chúng bằng trọng số lý tưởng, để tách “ghép sai” khỏi

“ghép đúng nhưng cân sai”. Nếu biến thể hai cải thiện nhiều hơn biến thể một, thì bài toán của bạn nằm ở chương 34 chứ không nằm ở chương này.

→ **trường hợp riêng của E/24** — đây đúng là *oracle* experiment của chương 24, áp cho một khối cụ thể; giá trị của nó vẫn là chặn trên công sức trước khi bỏ công sức ra

D. Ngân sách trên Jetson Orin

Ở 30 khung hình mỗi giây, ngân sách là 33 mili-giây một khung, và đó là ngân sách cho *tất cả*: rút trích, bấm, quản lý bản đồ cục bộ, và phần dành sẵn cho biến động. Cách kế toán thì gọn: liệt kê từng khoản đang tiêu, đo từng khoản ở phân vị 95 chứ không ở trung vị vì phân vị mới là thứ làm bỏ khung, cộng lại, rồi lấy trần trừ đi. Chương này không đưa một bảng số mẫu, vì một bảng số chưa đo trên máy của bạn chỉ tạo cảm giác an tâm sai và rồi sẽ bị trích lại như thể nó là số đo. Nhưng ba kết luận về *cấu trúc* của bảng đó thì đúng trước cả khi bạn đo, và chúng mới là phần đáng nhớ.

- **Phần dự phòng cho biến động là khoản bị ăn mất trước tiên.** Nó không có tên trong bất kỳ hồ sơ thời gian nào, nên nó bị tiêu mà không ai ghi sổ. Khi nó hết, triệu chứng không phải chậm đều mà là bỏ khung từng đợt — và bỏ khung từng đợt trông giống hết một lỗi thuật toán, nên nó ăn mất rất nhiều thời gian gỡ lỗi.
- **Một bộ đặc trưng nhẹ chỉ vừa nếu nó thay chứ không thêm.** Nếu bạn vẫn chạy ORB song song để đối chứng, hãy tính đó là hai khoản chứ không phải một, và đừng dùng lần đo có chạy song song ấy để kết luận về ngân sách lúc vận hành.
- **Một bộ ghép cặp đầy đủ chạy mỗi khung thì gần như chắc chắn vượt trần.** Đây là hệ quả của chi phí bậc hai ở mục 4B chứ không của một con số cụ thể nào, nên nó không đôi khi bạn đổi phần cứng; cái đổi chỉ là số điểm mà tại đó nó bắt đầu vượt. Lối thoát duy nhất là hạ tần suất chạy, và đó chính là lỗi hai ở mục 6F.

Ba điều nữa mà kế toán trên chưa thể hiện. Thứ nhất, mạng phải qua TensorRT với kích thước đầu vào cố định, và bước chuyển đổi này thường lộ ra các phép toán không được hỗ trợ. Thứ hai, GPU trên Orin đã bị dùng cho những việc khác, nên độ trễ đo lúc chạy một mình không phải độ trễ lúc chạy trong hệ thật. Thứ ba, độ trễ của mạng gần như hằng số, còn độ trễ của ORB thay đổi theo nội dung ảnh — đổi sang mạng làm hệ dễ dự đoán hơn, và đó là một lợi ích thật, ít khi được kể tới.

E. Tích hợp vào ORB-SLAM3 thì phải bỏ những gì

Đây là danh sách mà mọi bài báo đều bỏ qua, và là chỗ chi phí thật nằm. Thay bộ đặc trưng không phải thay một hàm; nó gỡ ra bốn thứ đã ăn sâu vào toàn hệ thống.

- **Từ điển túi từ.** DBow2 được huấn luyện trên bộ mô tả nhị phân ORB. Bộ mô tả số thực làm nó vô dụng ngay lập tức. Mất túi từ là mất cả lấy ứng viên đóng vòng lẫn định vị lại sau khi mất bấm. Bạn phải hoặc huấn luyện lại một từ điển mới, hoặc chuyển sang bộ mô tả toàn ảnh — và đó là nội dung của chương 29.
- **Ngưỡng Hamming đã tinh chỉnh nhiều năm.** TH_HIGH bằng 100 và TH_LOW bằng 50 không còn nghĩa gì với khoảng cách L2. Mỗi chỗ gọi phải suy lại ngưỡng, và trong ORB-SLAM3 có hàng chục chỗ gọi như vậy, mỗi chỗ dùng một *mfNNratio* riêng.
- **Bất biến tỉ lệ theo kim tự tháp — và đây là chỗ nguy hiểm nhất.** Tầng kim tự tháp của mỗi điểm không chỉ dùng cho việc phát hiện. Nó làm ba việc nữa. Nó đặt bán kính tìm kiếm khi chiếu điểm bản đồ xuống ảnh. Nó đặt kiểm tra nhất quán tỉ lệ khi quyết định một điểm bản đồ có quan sát được không. Và quan trọng nhất, nó đặt *ma trận thông tin của mọi cạnh tái chiếu* trong tối ưu, qua *mvInvLevelSigma2[octave]*. Một bộ phát hiện học được không trả về

tầng kim tự tháp. Nếu bạn điền một hằng số vào đó, bạn vừa đặt trọng số bằng nhau cho mọi phương trình trong bài toán tối ưu, một cách âm thầm. Thay frontend hoá ra là thay cả backend.

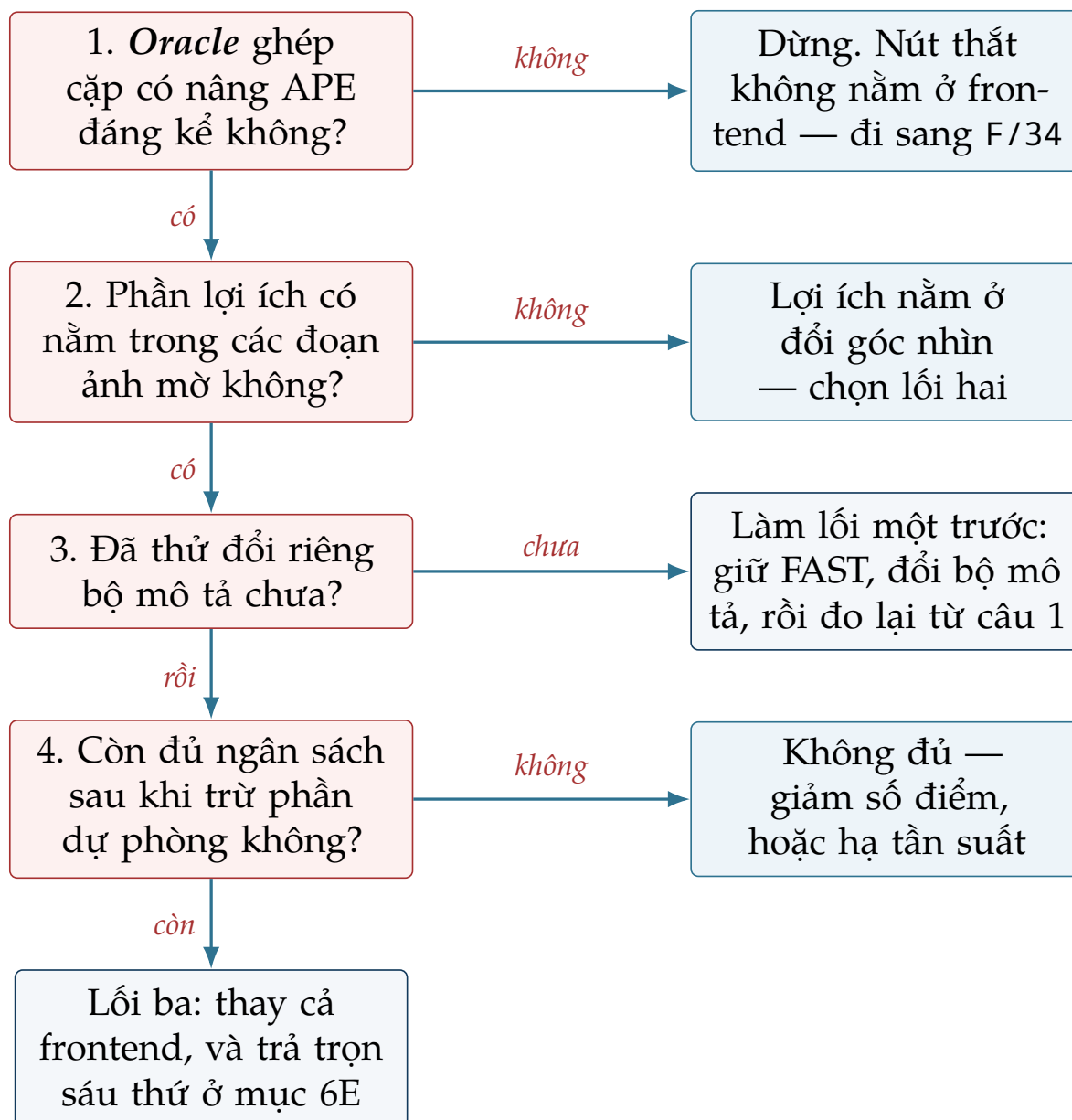
- **Bộ mô tả đại diện của điểm bản đồ.** MapPoint chọn bộ mô tả đại diện bằng cách lấy cái có khoảng cách Hamming trung vị nhỏ nhất tới các quan sát khác. Với vector số thực thì khái niệm đó phải định nghĩa lại, và trung bình không tương đương trung vị.
- **Phân bố đều theo cây tám nhánh.** DistributeOctTree ép các điểm trải đều trên ảnh. Cơ chế triệt phi cực đại của một mạng có hành vi khác hẳn. Nhớ lại cơ chế hấp thụ số hai: độ trải là thứ quyết định điều kiện của bài toán, nên đây không phải chi tiết nhỏ.
- **Các hằng số cố.** Số đặc trưng 1200, iniThFAST 12, minThFAST 4, và các cổng dạng “dưới 15 cặp thì coi như mất bám”. Tất cả đều được chỉnh cho phân phối số cặp ghép của ORB. Bộ đặc trưng mới cho phân phối khác, nên các cổng đó phải chỉnh lại từ đầu.

F. Ba lối đi giữ được phần lớn hệ thống

Danh sách trên nghe như một lời từ chối. Nó không phải. Nó chỉ nói rằng “thay ORB” là lựa chọn đắt nhất trong ba lựa chọn, và hai lựa chọn kia ít khi được nhắc tới.

- **Lối một — giữ nguyên bộ phát hiện, chỉ đổi bộ mô tả.** Vẫn chạy FAST trên kim tự tháp, vẫn có tầng, vẫn có phân bố đều theo cây tám nhánh, nhưng tính bộ mô tả bằng HardNet hoặc bằng phần mô tả của SuperPoint trên các ô ảnh đã cắt. Lối này giữ nguyên toàn bộ ba chức năng của tầng kim tự tháp, nên ma trận thông tin của backend không đổi. Nó chứa chế độ hồng số hai và một phần số ba, và không chứa số một. Chi phí thêm rất thấp, vì mạng chỉ chạy trên vài trăm ô ảnh nhỏ chứ không trên toàn ảnh. Theo mình đây là lối có tỉ lệ lợi ích trên rủi ro tốt nhất nếu bạn bắt buộc phải đổi một thứ ở mỗi khung hình.
- **Lối hai — giữ nguyên ORB, chỉ thêm một bộ ghép cặp học được ở tầng đóng vòng.** Không đụng gì tới đường bám khung hình. Bộ ghép cặp chạy ở cỡ 1 Hz, chỉ trên các cặp ứng viên đã qua bước lấy ứng viên. Nó nhắm đúng chế độ hồng số ba, và đó là chế độ mà bám khung hình không bao giờ gặp. Rủi ro thấp vì nếu nó hỏng thì bạn chỉ mất một vòng, không mất bám.
- **Lối ba — thay cả frontend.** Chỉ nên chọn khi *oracle* cho thấy trần rất cao và lối một đã được thử và không đủ. Khi đó bạn phải trả trọn danh sách sáu thứ ở mục trên, và phải tự đặt lại một mô hình bất định thay cho tầng kim tự tháp.

Hình 1.8 gói cả chương thành một cây quyết định. Nó chỉ có bốn câu hỏi, và ba trong bốn câu trả lời được bằng hai ngày làm việc chứ không bằng ba tháng.



Hình 1.8. Cây quyết định cho câu hỏi “có nên thay frontend không”. Ba trong bốn câu hỏi trả lời được bằng hai ngày đo đạc, và câu đắt nhất — câu số bốn — chỉ được hỏi sau khi ba câu kia đã cho phép. Chú ý hai nhánh sớm: nếu *oracle* cho trần thấp, hoặc nếu phần lợi ích không nằm trong các đoạn ảnh mờ, thì cả chương này không áp dụng cho hệ của bạn.

G. Dựng một cái cân của riêng bạn

Vấn đề với HPatches và các bảng xếp hạng là chúng đo sai điểm vận hành. Cách sửa không phải bỏ đo, mà là dựng một cái cân từ chính dữ liệu của bạn. Việc này tốn nửa ngày và dùng được mãi.

Nguyên liệu là một chuỗi EuRoC có chân trị tư thế và có ảnh stereo. Từ tư thế chân trị và độ sâu stereo, bạn dựng được cặp ghép đúng cho bất kỳ cặp khung hình nào: chiếu điểm từ khung i sang khung j , và chấp nhận nếu sai lệch dưới hai điểm ảnh. Đó là chân trị ghép cặp, không cần gán tay.

Điều quan trọng là *cách chọn cặp khung hình*, vì đó chính là chỗ các bảng xếp hạng sai. Hãy chia các cặp thành ba nhóm theo đúng ba chế độ vận hành của hệ bạn.

- **Nhóm tám** — cặp khung hình liên tiếp, cách nhau 33 mili-giây. Đây là tuyệt đại đa số các lần ghép cặp trong một lần chạy, nên nó phải là nhóm có trọng số lớn nhất khi bạn đọc bảng.
- **Nhóm mờ** — cặp khung hình liên tiếp nhưng lấy trong các đoạn có tốc độ góc cao nhất chuỗi, lọc từ IMU; ngưỡng thì lấy từ chính phân bố tốc độ góc của bạn, chẳng hạn phân vị trên cùng, chứ không lấy từ một con số cho sẵn.
- **Nhóm vòng** — cặp khung hình cách nhau trong thời gian nhưng gần nhau trong không gian, lấy từ chân trị. Đây là chế độ vận hành của đóng vòng.

Rồi đo cùng ba con số cho từng nhóm: số cặp ghép đúng, tỉ lệ inlier, và *độ trái* của các cặp ghép trên ảnh. Độ trái đo bằng trị riêng nhỏ nhất của ma trận hiệp phương sai toạ độ các điểm được ghép, hoặc đơn giản hơn là số ô còn có điểm khi chia ảnh thành lưới bốn nhân bốn.

Bảng ba nhóm nhân ba con số này nói được điều mà không bảng xếp hạng nào nói: phương pháp nào tốt hơn ở *nhóm nào*. Dự đoán của mình, và đây là dự đoán chứ không phải kết quả đo: các phương pháp học được sẽ hơn rất ít ở nhóm tám, hơn rõ ở nhóm mờ, và hơn nhiều ở nhóm vòng. Nếu bảng của bạn ra đúng hình dạng đó thì cây quyết định ở Hình 1.8 đưa bạn thẳng tới lối hai.

H. Điền bốn cổng của chương 27 cho ba họ

Gộp tất cả lại, Bảng 1.4 là câu trả lời của chương này cho khung bốn cổng. Đây là bảng nên chụp lại và dán lên tường.

Bảng 1.4. Bốn cổng của chương 27, điền cho ba họ phương pháp. Cột cuối là kết luận của mình tính tới tháng 9/2026, và nó có điều kiện: nó giả định bạn *đã* có một hệ stereo-inertial tinh chỉnh kỹ chạy trên Orin.

Họ	Bài toán có thật?	Đo được?	Vừa ngân sách Orin?	Kết luận
Bộ mô tả học được trên điểm ORB	Có: chế độ hồng đo đổi sáng, đo được bằng biên δ	Có: nhóm mờ và nhóm vòng của mục 6G	Nhiều khả năng vừa: mạng chỉ chạy trên vài trăm ô ảnh nhỏ	Đáng thử trước tiên
Đặc trưng học được thay cả FAST	Có, nhưng chỉ chứa hai trong bốn chế độ hồng	Có, nhưng phải đo lại toàn bộ chuỗi vì tăng kim tự tháp mất	Cần đo; XFeat và ALIKED là hai ứng viên duy nhất đáng thử	Chỉ khi <i>oracle</i> cho trần cao
Bộ ghép cặp học được mỗi khung	Bài toán gán toàn cục không phải nút thất lúc tám	Có, nhưng cải thiện dễ bị năm cơ chế hấp thụ	Gần như chắc chắn không, do chi phí bậc hai	Chưa đáng
Bộ ghép cặp học được ở tầng vòng	Có: đây đúng là chế độ hồng thứ ba	Có: đo bằng recall vòng, xem chương 29	Có, vì chỉ chạy ở cỡ 1 Hz trên vài cặp ứng viên	Đáng, và là phương án tốt nhất
Ghép dày không bộ phát hiện	Có: vùng ít vân là chế độ hồng thật	Khó, vì phải dựng thêm tầng danh tính mới đo được trong hệ	Không cho mỗi khung; có thể cho tầng vòng nếu tối ưu kỹ	Chưa đáng, trừ khi cảnh của bạn rất ít vân

Một lưu ý về cách đọc bảng này. Nó không nói các phương pháp ở hàng dưới kém hơn. Nó nói chúng được thiết kế cho một điểm vận hành khác, và ở điểm vận hành đó chúng thắng rõ. Nhầm hai câu này là nhầm phổ biến nhất khi đọc văn liệu ghép cặp.

CẦN NHỚ

Trước khi tích hợp bất cứ mạng nào, hãy chạy *oracle* ghép cặp: thay đầu ra của bộ ghép cặp bằng cặp ghép dựng từ chân trị, rồi đo APE. Con số đó là *trần* của mọi công sức frontend. Nếu trần chỉ hơn hiện tại vài phần trăm thì câu trả lời cho cả chương này là “chưa đáng”, và bạn biết điều đó sau hai ngày thay vì sau ba tháng. Nếu vẫn muốn đi tiếp, hãy đi theo

con đường rẻ nhất: giữ ORB cho bám khung hình ở 30 Hz, và chỉ đặt bộ ghép cặp học được ở tầng kiểm chứng đóng vòng, chạy ở cỡ 1 Hz.

1.7 Giới hạn và trường hợp hỏng

Giới hạn lớn nhất của chương này là giới hạn của bằng chứng. Gần như mọi con số so sánh đặc trưng đều đo trên cặp ảnh hai khung nhìn rộng, không có tiên nghiệm chuyển động, không có ràng buộc thời gian thực. Chưa có một nghiên cứu ablation nào đủ sạch trả lời câu “thay ORB bằng X thì APE trên EuRoC đổi bao nhiêu, với cùng ngân sách tính toán”. Vài công trình có báo cáo, nhưng chúng thường đổi nhiều thứ cùng lúc và không cố định ngân sách. Nên mọi kết luận trong chương này về APE là *suy luận từ cơ chế*, không phải kết quả đã đo, và mình muốn nói rõ điều đó.

Chế độ hỏng thứ hai là hỏng khi ra ngoài phân phối huấn luyện. Bộ đặc trưng học được mang theo giả định của dữ liệu huấn luyện nó. Nếu cảnh của bạn là hầm mỏ, ban đêm, hoặc ảnh hồng ngoại, thì mọi thứ hạng trên bảng xếp hạng ngoài trời ban ngày đều không chuyển sang được. Tệ hơn, kiểu hỏng cũng đổi: ORB hỏng bằng cách không cho điểm nào, còn mạng hỏng bằng cách cho ra các điểm tự tin nhưng sai. Kiểu hỏng thứ hai độc hơn nhiều với một đường ống có RANSAC, vì các cặp sai *nhất quán* với nhau thì RANSAC không loại được.

Chế độ hỏng thứ ba mang tính hệ thống. Chương này giả định frontend tách rời được khỏi phần còn lại, để có thể thay riêng nó. Mục 6E cho thấy giả định đó sai ở một chỗ cụ thể: tầng kim tự tháp là cầu nối giữa frontend và ma trận thông tin của backend. Khi một khối bị nối với khối khác qua một kênh không có tên trong tài liệu như vậy, ablation cho ra con số không có nghĩa như ta tưởng — đúng cảnh báo cuối chương 24.

Cuối cùng là giới hạn về tính chẩn đoán được. Khi ORB không tìm ra điểm, bạn mở ảnh lên và thấy ngay vì sao. Khi một mạng cho ra tập điểm kỳ lạ ở một cảnh cụ thể, bạn không có cách nào đi ngược từ triệu chứng về nguyên nhân trong vài giờ. Với một hệ đang chạy sản phẩm, mất khả năng chẩn đoán là một chi phí thật, và nó không xuất hiện trong bất kỳ bảng so sánh nào.

Giới hạn thứ năm là phạm vi mà chương này cố ý bỏ qua. Frontend của ORB-SLAM3 không chỉ ghép giữa hai khung hình liên tiếp. Nó còn ghép giữa hai ảnh stereo bằng ràng buộc đường epipolar, với tinh chỉnh dưới mức điểm ảnh dọc đường quét, để lấy độ sâu. Nó còn ghép giữa khung hình và bản đồ cục bộ theo phép chiếu. Ba bài toán ghép đó có ràng buộc khác nhau và điểm vận hành khác nhau. Một bộ đặc trưng học được phải làm tốt cả ba, nhưng mọi bảng xếp hạng chỉ đo bài toán thứ nhất. Đây là một khoảng trống bằng chứng nữa, và nó cùng loại với khoảng trống đã nói ở hộp cam bẫy.

Cuối cùng là một giới hạn về chính chương này. Nó viết theo giọng hoài nghi, và giọng đó phù hợp với một người *đã có* một hệ thống chạy tốt. Với một người đang xây từ đầu, cân nhắc đảo ngược: một đường ống dựa trên SuperPoint cộng LightGlue cho kết quả chấp nhận được nhanh hơn nhiều so với việc tinh chỉnh ORB trong hai năm. Kết luận “chưa đáng” của chương này là kết luận có điều kiện, và điều kiện đó là hệ thống hiện có của bạn.

1.8 Thực hành

Repo và tài nguyên. [kornia](#) có sẵn HardNet, DISK, LoFTR và LightGlue dưới một giao diện chung, nên nó là chỗ rẻ nhất để so nhiều phương pháp mà không dựng lại từng repo. [magicleap/SuperGluePretrainedNetwork](#) và [cvgl/LightGlue](#) cho hai bộ ghép cặp;

`cvg/Hierarchical-Localization` cho một đường ống hoàn chỉnh từ đặc trưng tới định vị; `verlab/accelerated_features` cho XFeat; `zju3dv/LoFTR` cho nhánh không bộ phát hiện. Về hạ tầng dài hạn thì COLMAP và `opencv` bền hơn hẳn mọi tên mô hình ở trên. Danh sách này chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; riêng bốn chế độ hồng ở mục 2 thì không cũ.

Câu hỏi kiểm tra hiểu. Viết công thức của độ dài vệt mờ tối hạn theo w, C, t , rồi nói ngưỡng tốc độ góc đổi theo hướng nào và theo tỉ lệ nào khi bạn hạ ngưỡng FAST xuống một nửa, và khi cảnh tối đi làm tương phản giảm một nửa? Nếu bạn giảm số điểm còn một phần ba thì chi phí của một bộ ghép cặp kiểu SuperGlue giảm mấy lần? Trong sáu thứ phải bỏ ở mục 6E, thứ nào chạm vào backend chứ không chỉ chạm vào frontend?

Mini project trong khối *duan bên dưới*. Hai dự án ở cuối chương nối tiếp nhau: một cái đo trần, một cái đo ngưỡng hồng. Chạy cái đo trần trước.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● FAST	Bộ phát hiện góc của ORB. Nó nhận một điểm là góc nếu có 9 điểm ảnh liên tiếp trên vòng tròn bán kính 3 cùng lệch quá ngưỡng t . Nó hồng khi ảnh mờ, vì trung bình theo hướng chuyển động kéo tương phản xuống dưới t và cắt đứt cung liên tiếp.
●●● BRIEF	Bộ mô tả nhị phân 256 bit, mỗi bit là kết quả một phép so sánh “điểm A sáng hơn điểm B”. Nó bất biến với đổi độ sáng đơn điệu toàn cục, nhưng hồng khi biên của các phép so sánh co lại và bit bắt đầu lật ngẫu nhiên.
●●● Biên δ của một phép so sánh	Chênh lệch cường độ thật của một cặp thử trong BRIEF. Xác suất lật bit là $\Phi(-\delta/\sigma\sqrt{2})$, nên chính δ chứ không phải độ sáng tuyệt đối quyết định bộ mô tả còn dùng được hay không.
●● MMA	Trong các cặp được ghép, tỉ lệ cặp có sai số vị trí dưới một ngưỡng điểm ảnh. Nó đo bộ mô tả cộng bộ ghép cặp, và nó là thước đo phổ biến nhất trên HPatches.
●● HardNet	Bộ mô tả học bằng mất mát triplet với mẫu âm khó nhất trong batch. Chọn mẫu âm khó nhất là cách xấp xỉ đúng đối thủ lúc chạy, tức ứng viên nhì trong phép thử tỉ lệ.
●●● Làm giàu bằng homography	Cách SuperPoint chế nhãn cho bộ phát hiện: áp hàng trăm homography ngẫu nhiên, phát hiện, ánh xạ ngược, rồi giữ các điểm sống sót. Giới hạn của nó là phân phối huấn luyện chỉ có biến dạng phẳng, không có ảnh mờ và không có thị sai.
●●● SuperPoint	Một mạng, hai đầu ra: bản đồ điểm và bộ mô tả 256 chiều, huấn luyện tự giám sát bằng homography. Nó là mặc định của cả dòng, nhưng chưa từng thấy ảnh mờ trong huấn luyện trừ khi bạn tự thêm vào.
●● R2D2	Tách tính lặp lại khỏi tính phân biệt thành hai bản đồ riêng. Lý do: góc của một viên gạch rất lặp lại nhưng ghép luôn sai, vì nó giống năm trăm viên khác.
● DISK	Coi chọn điểm và chấp nhận cặp ghép là hành động lấy mẫu, thường cặp đúng và phạt cặp sai, rồi học bằng <i>gradient</i> chính sách. Nó tối ưu thẳng mục tiêu rời rạc thay vì một hàm thay thế khả vi. DeDoDe đi hướng khác: tách hẳn việc huấn luyện bộ phát hiện khỏi bộ mô tả, để bộ phát hiện không bị kéo về phía sở thích của bộ mô tả.
●● XFeat	Đặc trưng thiết kế cho CPU và biên: mạng nông, chạy ở một phần tư độ phân giải, và tinh chỉnh vị trí bằng một mạng nhỏ chạy sau khi đã ghép cặp. Đây là công trình duy nhất trong dòng lấy ngân sách độ trễ làm ràng buộc thiết kế.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• ALIKED	Học các vị trí lấy mẫu biến dạng được cho vùng hỗ trợ của bộ mô tả, thay cho ô vuông cố định. Nó nhắm thẳng chế độ hồng do đối góc nhìn, vì ô vuông cố định méo khi mặt phẳng nghiêng.
••• SuperGlue	Bộ ghép cặp dùng chín tầng chú ý xen kẽ trong và giữa hai ảnh, rồi giải bài toán gán bằng Sinkhorn có thùng rác. Nó đổi quyết định cục bộ lấy quyết định toàn cục, và trả bằng chi phí bậc hai theo số điểm.
••• LightGlue	Cùng ý tưởng SuperGlue nhưng rút ngắn ba chỗ: thoát sớm khi cặp ảnh dễ, cắt bỏ điểm không ghép được giữa chừng, và bỏ vòng lặp Sinkhorn. Cắt điểm là chỗ tiết kiệm lớn nhất vì chi phí là bậc hai.
••• LoFTR	Ghép không cần bộ phát hiện: ghép dày ở một phần tám độ phân giải bằng chú ý tuyến tính, rồi tinh chỉnh dưới mức điểm ảnh trong cửa sổ nhỏ. Nó cứu vùng ít vân, nhưng chỉ ghép được hai ảnh một lúc và không để lại bộ mô tả nào để lưu.
••• Oracle ghép cặp	Thay đầu ra bộ ghép cặp bằng cặp ghép dựng từ chân trị rồi đo APE. Kết quả là trần của mọi cải tiến frontend, và nó tốn hai ngày thay vì ba tháng.
••• Vì sao chi phí SuperGlue là bậc hai?	Vì mỗi điểm phải nhìn mọi điểm khác, nên ma trận điểm số có N^2 ô. Kế toán thô là $36N^2d$ phép nhân cộng cho chín tầng, tức nhân đôi số điểm thì chi phí gấp bốn.
••• Vì sao bỏ bộ phát hiện cứu được vùng ít vân?	Vì nó không làm tường có vân, mà đổi loại bằng chứng được dùng. Chồng tầng chú ý đưa thông tin từ mép tường vào ô ở giữa tường, nên cặp ghép được quyết bằng bằng chứng phi cục bộ thay vì cục bộ.
••• Vì sao thay bộ phát hiện lại là thay cả backend?	Vì tầng kim tự tháp không chỉ dùng để phát hiện. Nó đặt bán kính tìm kiếm, kiểm tra nhất quán tỉ lệ, và đặt ma trận thông tin của mọi cạnh tái chiếu qua $mvInvLevelSigma2$. Bộ phát hiện học được không trả về nó, nên thay frontend là thay luôn trọng số của backend.
••• Tỉ lệ inlier tăng mạnh mà APE không đổi — nghĩa là gì?	Nghĩa là frontend không phải ràng buộc chặt. Năm cơ chế hấp thụ: bão hoà sau vài chục cặp trái đều, phân bố quan trọng hơn số lượng, bộ làm bền đã lọc rồi, APE do đuôi phân phối quyết định, và điểm vận hành khác với bảng xếp hạng.
••• Vào hành lang tối, tracking vẫn bám nhưng bản đồ ngừng lớn — nghĩa là gì?	Khoảng cách Hamming của cặp ghép đúng đã trôi vào khoảng giữa TH_LOW và TH_HIGH . Cặp ghép còn qua được ngưỡng lỏng của bám khung hình, nhưng rớt ở ngưỡng chặt của tam giác hoá điểm mới.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
kornia	Hiện thực chung của HardNet, DISK, LoFTR, LightGlue dưới một giao diện — chỗ so sánh rẻ nhất	Hạ tầng; bền hơn tên mô hình
hloc	SuperPoint cộng SuperGlue hoặc LightGlue trong một đường ống định vị hoàn chỉnh	Đây là nơi dòng này thật sự được dùng trong sản phẩm
COLMAP với đặc trưng học được	Ghép cặp khung nhìn rộng để tái dựng ba chiều — chỗ ghép cặp học được thắng rõ ràng nhất	Không có ràng buộc thời gian thực, nên đúng điểm vận hành
Image Matching Challenge [20]	Bảng xếp hạng quyết định thứ hạng của cả dòng này	Các giải pháp thắng thường là tổ hợp nhiều mô hình, xa mức thời gian thực

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Chính ORBmatcher của bạn	Phép thử tỉ lệ và biểu đồ nhất quán góc là hai bản thô của đúng hai ràng buộc mà SuperGlue đưa vào trong mô hình	Ví dụ nội bộ; đọc lại mã theo góc nhìn này rất đáng

MINI PROJECT SLAM · Đo trần của frontend bằng *oracle* ghép cặp · 1–2 ngày

Mục tiêu	trả lời dứt điểm câu “đổi bộ đặc trưng thì APE tốt lên nhiều nhất là bao nhiêu”, trước khi tích hợp bất cứ mạng nào.
Dữ liệu	hai chuỗi EuRoC, một dễ và một khó có chuyển động nhanh (chẳng hạn V2_03), cùng chân trị tư thế.
Các bước	dựng một chế độ <i>oracle</i> trong ORBmatcher: với mỗi điểm bản đồ, chiếu nó bằng tư thế chân trị và nhận cặp ghép nếu điểm đặc trưng gần nhất nằm trong bán kính hai điểm ảnh, bỏ qua hoàn toàn khoảng cách mô tả. Chạy ba lần mỗi cấu hình để có sần nhiễu. So ba cấu hình: ORB thường, <i>oracle</i> toàn chuỗi, và <i>oracle</i> chỉ bật trong các đoạn có tốc độ góc cao nhất chuỗi — ngưỡng thì lấy từ dự án thứ hai bên dưới, tức là từ số đo của chính bạn.
Xong khi	bạn có một bảng ba dòng kèm độ lệch chuẩn, và phát biểu được “trần của mọi cải tiến frontend trên chuỗi này là X cm”. Kiểm tra chéo: hiệu giữa dòng hai và dòng ba cho biết bao nhiêu phần của trần đó nằm riêng trong các đoạn chuyển động nhanh.
Nối với	E/24 cho cách đọc con số và cách tính sần nhiễu; F/29 sẽ dùng lại đúng chế độ <i>oracle</i> này ở tầng đóng vòng.

MINI PROJECT SLAM · Tìm ngưỡng mờ giết FAST trên chính chuỗi của bạn · nửa ngày

Mục tiêu	biến công thức $d^* = wC/t$ ở mục 2A thành một con số của bạn, đo trên dữ liệu thật, rồi biến nó thành một cờ chặn đoán chạy trực tuyến.
Dữ liệu	một chuỗi EuRoC có IMU, cộng nhật ký số điểm FAST phát hiện được mỗi khung hình.
Các bước	ghi hai chuỗi số theo thời gian: chuẩn của tốc độ góc từ IMU, và số điểm đặc trưng rút được. Vẽ số điểm theo tốc độ góc dưới dạng biểu đồ tán xạ. Lập lại với iniThFAST bằng 20 và bằng 12 để thấy đường cong dịch chuyển. Song song đó, tự tính dự đoán của mô hình: đo w và C trên vài ô ảnh điển hình của chuỗi, lấy tiêu cự từ file hiệu chỉnh và thời gian phơi sáng từ nhật ký camera, rồi ráp vào $\omega \approx (wC/t)/(f \Delta t)$.
Xong khi	bạn đọc ra được từ biểu đồ một ngưỡng tốc độ góc mà quá đó số điểm sụp, và bạn có một con số dự đoán từ mô hình để so với nó. Hai con số đều là của bạn, không mượn của ai. Kiểm tra chéo: hạ ngưỡng FAST một nửa thì ngưỡng tốc độ góc đo được phải dịch theo đúng hướng mà công thức nói, và dịch cỡ đúng tỉ lệ; nếu không thì mô hình một chiều đang thiếu một cơ chế, và việc tìm ra cơ chế đó đáng giá hơn cả con số.
Nối với	nối thẳng vào dự án trên: các đoạn vượt ngưỡng chính là các đoạn cần bật <i>oracle</i> riêng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Đại lượng bị mờ tấn công là tương phản cục bộ còn lại. Mô hình trung bình một chiều cho $C_{\text{hiệu dụng}} \approx C \min(1, w/d)$, nên FAST mờ khi vệt mờ đạt $d^* \approx wC/t$. Đổi sang tốc độ góc bằng $\omega \approx (d^*/f)/\Delta t$, với f là tiêu cự tính theo điểm ảnh và Δt là thời gian phơi sáng. Ba tham số quyết định là ngưỡng FAST, tiêu cự và thời gian phơi sáng; thay số của chính bạn vào thì ra ngưỡng của bạn. Ở đây không đưa con số vì nó chưa được đo trên hệ của bạn.
- A2. Vì thứ giết BRIEF không phải giá trị mà là biên, và chính xác hơn nữa là tỉ số giữa biên và nhiễu. Xác suất lật một bit là $\Phi(-\delta/\sigma\sqrt{2})$, nên Hamming kỳ vọng của cặp đúng là $256 \Phi(-\delta/\sigma\sqrt{2})$. Khi δ/σ tụt về cùng bậc với 1, hàm Φ vào vùng dốc nhất và Hamming nhảy vọt — đủ để vượt TH_LOW trong khi vẫn dưới TH_HIGH, tức là qua cửa lỏng nhưng rớt ở mọi cửa chặt.

- A3. Vì tiên nghiệm biến bài toán gán toàn cục thành bài toán gán trong một cửa sổ hẹp: mỗi điểm chỉ còn một nhóm ứng viên, nên chi phí từ N^2 xuống $N \cdot k$ với k là số ứng viên trong cửa sổ, gần như tuyến tính. Không bài báo nào tận dụng vì bảng xếp hạng của họ đo cặp ảnh hai khung nhìn rộng, nơi không tồn tại tiên nghiệm nào để tận dụng.
- A4. Vì năm cơ chế hấp thụ nằm giữa frontend và APE: bảo hoà sau vài chục cặp trải đều, phân bố quan trọng hơn số lượng, các bộ làm bền đã lọc phần lớn rồi, APE do vài giây tệ nhất quyết định trong khi ba thước đo kia là trung bình, và điểm vận hành của bảng xếp hạng khác hẳn của bạn.
- A5. Vì bỏ bộ phát hiện thì mất bộ mô tả, mà mất bộ mô tả thì mất danh tính của điểm. SLAM cần ghép một khung mới với hàng vạn điểm bản đồ, tức một-nhiều; ghép dày chỉ làm được ảnh với ảnh. Hệ quả kèm theo: túi từ nhị phân không dùng được nữa, nên mất luôn định vị lại và lấy ứng viên đóng vòng.

NÓI LẠI THẬT ĐƠN GIẢN

Hai người muốn gặp nhau ở một góc phố nhưng chỉ nói chuyện được qua điện thoại. Cách cũ: mỗi người tự ghi lại hai trăm năm mươi sáu điều nhỏ về chỗ mình đứng, kiểu “viên gạch bên trái sẫm hơn viên bên phải”, rồi hai bên đọc danh sách và so xem có khớp không. Cách này chạy rất nhanh và rất tốt, miễn là trời sáng và hai người đứng yên. Nếu có sương mù, hoặc nếu ai đó vừa chạy vừa nhìn, thì mọi điều nhỏ đều trở nên nửa vờ, và hai danh sách toàn điều nửa vờ thì khớp với mọi góc phố trong thành phố.

Cách thứ hai: đừng ghi riêng, hãy nói chuyện với nhau. “Tôi thấy một mái hiên đỏ bên trái, bạn có thấy không?” Nhờ trao đổi mà từng chi tiết mơ hồ được đặt vào ngữ cảnh, nên vẫn phân biệt được viên gạch này với viên gạch kia. Cái giá là cuộc gọi tốn thời gian, và thời gian tăng theo bình phương số chi tiết mỗi bên đem ra bàn.

Cách thứ ba: bỏ hẳn việc chọn ra vài chi tiết, so nguyên cả khung cảnh. Cách này ăn được cả bức tường trắng trơn. Nhưng nó không để lại một cuốn sổ nào, nên ngày mai muốn tìm lại đúng góc phố ấy thì phải gọi điện lại từ đầu, với từng người một.

Chỗ không rút gọn được: chuyện cái thước ở đầu ống lại quyết định trọng số ở cuối ống. Trong hệ thật, việc một điểm được nhìn thấy ở mức phóng to nào cũng chính là thứ quyết định người ta tin điểm đó tới đâu khi ghép mọi thứ lại. Đối cách chọn điểm mà quên điều này thì bạn vô tình tuyên bố mọi quan sát đáng tin như nhau, và không có cách nào nói điều đó bằng ngôn ngữ đời thường mà không mất đúng phần quan trọng.

Một câu: *Đặc trưng học được chứa được hai trong bốn chỗ ORB gây, nhưng chỗ nó chứa lại hiếm khi là chỗ quỹ đạo của bạn đang hỏng — nên hãy đo trần trước, rồi mới tiêu tiền.*

1.9 Kết nối

F/27-dat-hoc-sau-vao-dau là tiền đề trực tiếp: chương này là lần đầu tiên khung bốn cổng được điền đầy đủ cho một khối cụ thể, và kết quả điền ra là “chưa đáng cho mỗi khung hình, đáng cho tầng đóng vòng”. F/29-dinh-vi-lai-va-dong-vong nhận đúng kết luận đó làm điểm xuất phát, và trả lời câu hỏi mà chương này để lại: nếu đặt bộ ghép cặp học được ở tầng đóng vòng thì tầng nào của đóng vòng thật sự được lợi. F/30-do-sau-dong-quang-va-mo-hinh-nen là nhánh đối lập: thay vì ghép cặp thừa tốt hơn, nó bỏ ghép cặp và dùng dòng quang dày — cùng nỗi đau chuyển động nhanh, cách tiếp cận ngược lại. F/34-imu-bat-dinh-va-trien-khai nhận phần chương này cố ý để lại: nếu *oracle* cho thấy vấn đề nằm ở *trọng số* chứ không ở *cặp ghép*, thì lời giải nằm ở đó. E/24-thi-nghiem-va-ablation là công cụ của cả chương, vì *oracle* experiment là thứ duy nhất biến câu hỏi “có đáng không” thành một con số. C/16 giải thích cơ chế *attention* mà SuperGlue và LoFTR đều dựa vào.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ảnh mờ và đối độ sáng cần hai loại thuộc khác nhau, dù triệu chứng bề ngoài đều là “ít cặp ghép”?
2. Mô hình một chiều dự đoán một ngưỡng vết mờ $d^* = wC/t$, nhưng thực tế FAST chết sớm hơn ngưỡng đó — cơ chế nào giải thích phần chênh, và nó ảnh hưởng thế nào tới điều kiện của bài toán tư thế?
3. SuperPoint được huấn luyện bằng homography ngẫu nhiên. Điều đó khiến nó *không* học được ba hiện tượng nào, và hiện tượng nào trong ba cái đó là nỗi đau số một của bạn?
4. Nếu bạn chỉ được giữ một trong hai — ngữ cảnh toàn cục của SuperGlue hay ràng buộc một-một của Sinkhorn — thì giữ cái nào, và trên loại cảnh nào lựa chọn đó đảo ngược?
5. Tính lặp lại, MMA và tỉ lệ inlier cùng tăng mạnh mà APE không đổi: cơ chế nào trong năm cơ chế hấp thụ bạn kiểm chứng được rẻ nhất, và bằng phép đo gì?
6. Vì sao việc bộ phát hiện học được không trả về tầng kim tự tháp lại là một thay đổi ở *backend* chứ không chỉ ở *frontend*?
7. Nếu *oracle* ghép cặp cho thấy trần chỉ tốt hơn hiện tại vài phần trăm, bạn kết luận gì, và bạn ghi lại kết luận đó kèm những điều kiện nào để nó còn dùng được sau sáu tháng?

Nguồn

- [1] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, 2006.
- [2] M. Calonder, V. Lepetit, C. Strecha, et al., “BRIEF: Binary Robust Independent Elementary Features,” in *European Conference on Computer Vision (ECCV)*, 2010.
- [3] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *International Conference on Computer Vision (ICCV)*, 2011.
- [4] K. Mikolajczyk, T. Tuytelaars, and C. Schmid, “A Comparison of Affine Region Detectors,” *International Journal of Computer Vision*, vol. 65, no. 1–2, pp. 43–72, 2005.
- [5] A. Mishchuk, D. Mishkin, F. Radenović, et al., “Working Hard to Know Your Neighbor’s Margins: Local Descriptor Learning Loss,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [6] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018.
- [7] J. Revaud, P. Weinzaepfel, C. de Souza, et al., “R2D2: Repeatable and Reliable Detector and Descriptor,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [8] A. Barroso-Laguna, E. Riba, D. Ponsa, et al., “Key.Net: Keypoint Detection by Handcrafted and Learned CNN Filters,” in *International Conference on Computer Vision (ICCV)*, 2019.
- [9] M. J. Tyszkiewicz, P. Fua, and E. Trulls, “DISK: Learning Local Features with Policy Gradient,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [10] J. Edstedt, G. Bökman, M. Wadenbäck, et al., “DeDoDe: Detect, Don’t Describe — Describe, Don’t Detect for Local Feature Matching,” in *International Conference on 3D Vision (3DV)*, 2024.
- [11] G. Potje, F. Cadar, A. Araujo, et al., “XFeat: Accelerated Features for Lightweight Image Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [12] X. Zhao, X. Wu, W. Chen, et al., “ALIKED: A Lighter Keypoint and Descriptor Extraction Network via Deformable Transformation,” *IEEE Transactions on Instrumentation and Measurement*, vol. 72, 2023.
- [13] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [14] K. M. Yi, E. Trulls, V. Lepetit, et al., “LIFT: Learned Invariant Feature Transform,” in *European Conference on Computer Vision (ECCV)*, 2016.
- [15] M. Dusmanu, I. Rocco, T. Pajdla, et al., “D2-Net: A Trainable CNN for Joint Description and Detection of Local Features,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [16] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [17] M. Cuturi, “Sinkhorn Distances: Lightspeed Computation of Optimal Transport,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.

- [18] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [19] A. Katharopoulos, A. Vyas, N. Pappas, et al., “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” in *International Conference on Machine Learning (ICML)*, 2020.
- [20] Y. Jin, D. Mishkin, A. Mishchuk, et al., “Image Matching Across Wide Baselines: From Paper to Practice,” *International Journal of Computer Vision*, vol. 129, pp. 517–547, 2021.
- [21] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [22] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [23] J. Edstedt, Q. Sundfeldt, M. Wadenbäck, et al., “RoMa: Robust Dense Feature Matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [24] Y. Wang, X. He, S. Peng, et al., “Efficient LoFTR: Semi-Dense Local Feature Matching with Sparse-Like Speed,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [25] V. Balntas, K. Lenc, A. Vedaldi, et al., “HPatches: A Benchmark and Evaluation of Hand-crafted and Learned Local Descriptors,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.